ENGINEERING REFERENCE / EDITION 1.0

End-to-End AI System Design Handbook

Enterprise architecture from request to reliable outcome

Scalable serving. Permissioned knowledge. Bounded agents.

Security, evaluation, human review, and production operations.

Implementation-oriented reference. Numerical targets and prices are illustrative; validate them against real workloads and approved policies. Code and configuration are reference sketches, not a complete runnable service. Primary references checked on 10 October 2026.

20 CHAPTERS / 9 ARCHITECTURE & ER DIAGRAMS

Foundations for new engineers. Design depth for technical leaders.

10 October 2026

VENDOR-NEUTRAL REFERENCE

Contents

1. Executive summary and reading paths 3

2. Foundations and reference architecture 4

3. API gateway and edge layer 7

4. Input guardrails and security 9

5. Caching layers and invalidation 11

6. Intent routing, orchestration, and agents 14

7. RAG ingestion and data pipelines 17

8. Retrieval and grounded answer construction 21

9. Output guardrails and safe delivery 24

10. Human-in-the-loop operations 26

11. Load balancing, capacity, and cost 29

12. Monitoring, observability, and evaluation 31

13. Data model, contracts, and state 33

14. Deployment, security governance, and recovery 37

15. Reference implementation patterns 39

16. Testing, release gates, and rollout 43

17. Failure playbooks and architecture decisions 45

18. Best practices and delivery roadmap 47

19. Glossary 49

20. References and source notes 50

1. Executive summary and reading paths

An enterprise AI system is a distributed application that makes probabilistic model behavior useful inside deterministic boundaries. The model generates candidates. Identity services determine who is acting; authorization determines what they may access; application policy controls permissible actions; retrieval supplies evidence; validation decides what can be released; and operations keep the entire path measurable and recoverable. A more capable model does not remove these responsibilities.

The reference system in this handbook is a multi-tenant enterprise knowledge assistant. It answers questions from permissioned documents, supports approved read-only business tools, and can propose consequential actions for human review. Most requests use retrieval and one model call. Long-running research and actions use durable jobs. This running example makes the interactions among caching, RAG, tools, approval, and observability concrete.

The architectural position

- **Enforce authorization outside the model.** A system prompt, a retrieved document, or a model-generated tenant identifier cannot grant permission. Derive tenant and principal from verified credentials and enforce resource permissions again at retrieval, tool execution, and response release.
- **Keep expensive work bounded.** Set request deadlines, input and output limits, tool-call limits, concurrency quotas, and a monetary budget. Budget retries and fallback attempts within the original request envelope.
- **Treat every external text as data.** Uploaded files, search results, conversation history, tool responses, and model output can carry adversarial instructions. Limit the consequences of a successful injection with least privilege and controlled egress.
- **Maintain a chain of evidence.** Record document and chunk versions, prompt and model versions, policy decisions, tool effects, and release checks. Retain sanitized evidence sufficient to investigate an outcome without turning logs into a second sensitive data lake.
- **Separate correctness, safety, and availability.** A fast HTTP 200 can contain an unsupported answer. A correct refusal can be a good outcome. A timeout hidden behind a generic refusal is still an operational failure.
- **Use risk-based automation.** Low-risk answers can be automatic. Irreversible or consequential actions require narrowly scoped authorization and, where policy requires it, approval bound to the exact action.

How to use the handbook

Reader	Start here	Practical exercise
New engineer	Chapters 2-4, 7-9, and 19	Trace one question from login to cited answer; identify each trust boundary.
Backend or ML engineer	Chapters 5-13 and 15-16	Implement tenant-aware retrieval, request budgets, and an evaluation dataset.
Senior engineer or architect	Chapters 2, 5-6, and 11-17	Challenge consistency, cost, recovery, and failure assumptions with load and fault tests.
Security or operations lead	Chapters 4, 9-10, and 12-17	Verify revocation, approval, incident handling, and audit evidence end to end.

What production readiness means

A production candidate has a documented threat model, measurable service objectives, owned data sources, reproducible releases, enforced budgets, tested failure paths, and an operator who can stop harmful behavior. The strongest design is often a small deterministic workflow around a model, expanded only when evaluated requirements justify more autonomy. Enterprise governance should maintain risk ownership and evidence across the lifecycle; NIST's Generative AI Profile provides a complementary risk-management reference [1].

2. Foundations and reference architecture

2.1 The minimum vocabulary

A **token** is a unit of text or other model input/output used by a tokenizer; it is not a fixed number of characters or words. A **context window** limits what the model can process in a call. An **embedding** maps content to a numeric vector for similarity search. A **vector index** accelerates nearest-neighbor lookup, usually with a recall-versus-latency trade-off. **RAG** retrieves evidence at request time and supplies it to generation. An **agent** uses a model to choose some steps or tools; a **workflow** specifies those steps in application logic. Neither embeddings nor agent frameworks inherently enforce security.

Distinguish **training**, **fine-tuning**, and **inference**. Training changes model parameters. Fine-tuning adapts behavior with curated examples. Inference uses an existing model. RAG updates external knowledge without changing model weights. Use retrieval for rapidly changing facts and per-user access control; consider fine-tuning for persistent output style or task behavior after establishing a strong baseline and evaluation process. Fine-tuning is not a reliable permission boundary or a dependable deletion mechanism for memorized facts.

2.2 Three planes, separate responsibilities

The **serving plane** handles interactive requests, retrieval, inference, and approved tool calls. The **data plane** ingests documents and database changes, builds indexes, and propagates deletes and permission changes. The **control plane** manages policies, configuration, registries, evaluation gates, and rollouts. Isolate privileged control-plane writes from serving identities. A compromised assistant should not be able to change its own tool permissions or deploy a new prompt.

Figure 1. Reference architecture.

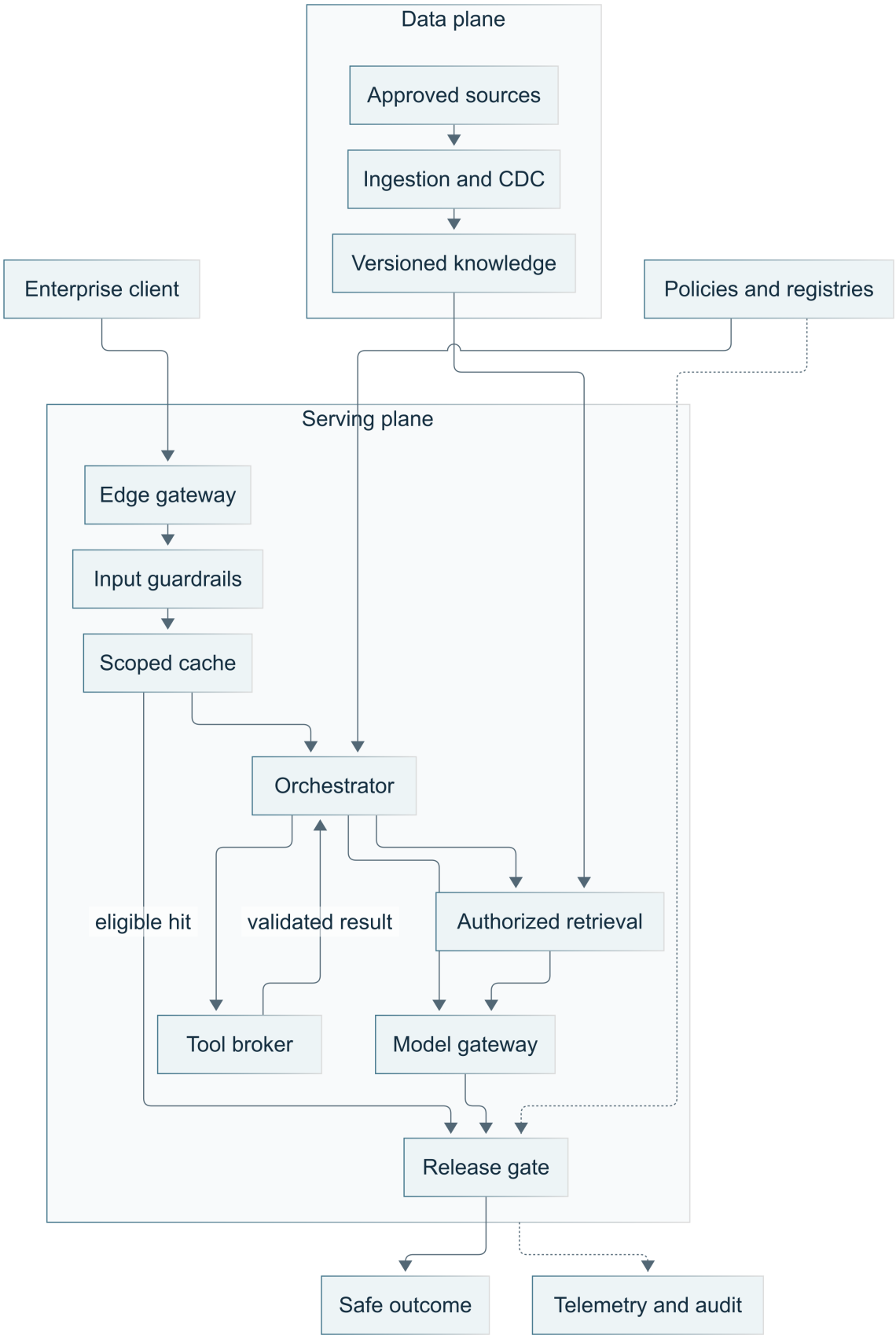


Figure 1. Reference architecture. Solid arrows carry execution or data; dotted arrows show policy or telemetry. Cache hits still pass the release gate. The tool loop is bounded and persisted.

2.3 Trust boundaries and invariants

The public edge, tenant-scoped application services, external model providers, data connectors, and tool systems are separate trust domains. Use short-lived workload identity between services, transport encryption, authenticated queues, and restrictive network egress. Provider routing must respect data classification, residency, and approved processing terms for every attempt, including fallback.

Four invariants simplify reviews: no unauthorized resource enters model context; no model output directly becomes an executable command; no unvalidated draft reaches a protected response channel; and no consequential action executes without an authoritative policy decision. These are design requirements that must be tested, not claims that an AI classifier can guarantee.

2.4 Running workload and service objectives

Assume 100 tenants, 1 million requests per day, a 200 requests/second burst, a 30-day reporting window, and a corpus of 1 million document versions. The assistant handles public FAQs, internal policy questions, and business-tool requests. These are planning assumptions, not observed benchmarks. Classify routes by risk and assign separate objectives; a buffered, evidence-checked answer has a different latency profile from a live token stream.

Objective	Illustrative target	Measurement and boundary
Interactive availability	99.9% over 30 days	Eligible requests returning a valid answer, valid refusal, or requested job receipt before deadline. Track each outcome separately.
Low-risk streaming responsiveness	p95 first released token below 2.5 s	Client-observed; excludes high-risk buffered responses by a declared route label.
Buffered answer latency	p95 below 12 s for short answers	Client-observed completed and validated response; output length distribution is fixed for the test.
Knowledge freshness	99% of accepted updates visible within 5 min	Source commit to query-visible generation, including queue and indexing time.
Permission revocation	No new release authorization after authoritative revocation	Serialize release decisions against current entitlements; already delivered or in-flight bytes cannot be retracted. Index cleanup is asynchronous.
Action safety	Zero unauthorized actions tolerated	Hard release gate and incident criterion, not a statistically proven zero-risk guarantee.

Define denominators before collecting metrics. Do not exclude provider failures, budget exhaustion caused by bad planning, or infrastructure overload simply because they produce a user-friendly message. Separate malformed and unauthorized requests from legitimate demand, while keeping independent abuse and rejection metrics. Service-level indicators and error budgets should reflect user outcomes [9].

2.5 Build the simplest deployment that meets the boundaries

Start with a modular application, one durable queue, an authoritative relational store, object storage, and a retrieval index. Logical components do not all need to be microservices. Split deployments when isolation, scaling profiles, ownership, or availability justify it: GPU serving, CPU-heavy parsing, privileged tools, and review workers commonly merit separation. Avoid making the router a distributed dependency chain before traffic or organizational needs require it.

3. API gateway and edge layer

3.1 Authenticate identity; authorize intent and resources

Use enterprise OIDC for user authentication and OAuth access tokens for API authorization. For browser applications use an appropriate authorization-code flow with PKCE and secure session handling; follow OAuth security best current practice [3]. Verify issuer, audience, allowed signing algorithms, signature, expiry, and token type. Reject tokens issued for another API. Plan signing-key rotation and bounded JWKS refresh so an identity-provider outage cannot turn into an authentication bypass.

The gateway derives the tenant from validated identity and a server-side membership mapping. If the client supplies a tenant selector, verify membership; never trust an arbitrary `X-Tenant-ID`. Propagate a signed internal context containing principal, tenant, scopes, authentication time, request ID, and policy version. Downstream services validate it or use workload-authenticated transport with an explicit trust contract. Resource authorization remains downstream because the edge cannot know every retrieved document or proposed tool argument.

Use service identities for machine clients, scoped API keys only where appropriate, and short-lived credentials wherever possible. Protect browser cookies with Secure, HttpOnly, and suitable SameSite settings; apply CSRF defenses when cookies authorize mutations. CORS configuration is not authentication. Authorize job polling, document citation downloads, review cases, and conversation access as rigorously as answer creation.

3.2 Rate limiting is multidimensional

Rate limiting caps work admitted in a time window. **Throttling** slows work or constrains concurrency. **Load shedding** rejects work when the service cannot safely process it. One requests-per-minute limit is insufficient because a 50-token question and a 50,000-token analysis consume very different resources.

Apply limits by IP at the unauthenticated edge and by tenant, user, route, provider, and model after identity resolution. Track requests, estimated tokens, reserved cost, simultaneous streams, queued jobs, uploaded bytes, and tool actions. Reserve estimated capacity before starting; reconcile actual usage afterward. Account for tokenizer estimation error and billable work that continues after disconnect.

Mechanism	Strength	Failure mode or trade-off
Token bucket	Allows controlled bursts with a sustained rate	Burst capacity can overwhelm a downstream dependency unless paired with concurrency limits.
Sliding window	More accurate recent-rate enforcement	Shared counters add storage and coordination cost.
Concurrency semaphore	Protects active streams and slow providers	Leaked permits after crashes require leases and recovery.
Bounded fair queue	Absorbs short bursts while separating tenants	Queuing increases latency; unlimited queues merely postpone failure.
Cost reservation	Prevents parallel requests exceeding a monetary budget	Reservations need atomicity, expiry, and reconciliation for uncertain provider outcomes.

Use a shared admission service or a partitioned quota design with explicitly bounded overshoot. Per-pod counters do not implement a fleet-wide quota. During quota-store failure, choose a documented mode: reject expensive requests or apply conservative leased local allowances. Never silently grant unlimited generation. Return `429` with retry guidance for quota rejection and `503` for temporary service capacity failures where appropriate.

3.3 Routing and request shaping

The edge routes to a regional application entry point according to tenant placement and approved residency. The application router later chooses the execution plan and model. Separate these concerns. Validate content type, schema version, body size, attachment count, decompressed size, and deadlines before expensive processing. Strip user-supplied internal headers. Establish correlation IDs without accepting arbitrary trace data as authorization.

Use distinct endpoints for interactive answers and durable jobs. For example, `POST /v1/answers` may stream or return a completed response; `POST /v1/jobs` returns `202` and a job ID. A receipt means accepted work, not a successful answer. Use stable error codes such as `AUTH_REQUIRED`, `QUOTA_EXCEEDED`, `POLICY_BLOCKED`, and `DEPENDENCY_UNAVAILABLE`; redact internal provider details from user-facing errors.

3.4 Streaming, cancellation, and idempotency

Server-Sent Events are often sufficient for one-way progress and tokens; WebSockets suit bidirectional sessions. Bound connection time and idle duration, send appropriate keepalives, propagate cancellation, and disable reverse-proxy buffering where intended. Do not allow CDN caching of personalized responses. After response headers are sent, report terminal stream failure in the stream protocol because an HTTP status can no longer convey it.

Use an idempotency key for durable jobs and mutations. Scope it to tenant, principal, operation, and a canonical payload hash. A repeated key with a different payload is a conflict. Persist the operation result or job reference and enforce uniqueness transactionally. A response cache is not an idempotency ledger. Reconnecting a stream should resume retained safe events or attach to the existing job, not execute a payment or ticket update again.

Acceptance evidence: load tests prove fair admission under one noisy tenant; identity tests reject audience and tenant spoofing; clients receive stable retry semantics; disconnect tests reclaim resources; duplicate mutation requests create one business effect.

4. Input guardrails and security

4.1 Order controls by cost and authority

Run cheap deterministic checks first: identity, size, schema, field constraints, MIME signatures, and known secret patterns. Then apply task-aware privacy and content policies, followed by more expensive classifiers if needed. Guardrails return structured decisions such as allow, transform, reject, or review, with policy version and reason code. Keep raw content out of ordinary logs even when validation fails.

PII means personally identifiable information, but sensitivity is broader: access tokens, private keys, trade secrets, internal financial information, regulated records, and confidential source code need protection too. Define a data-classification policy for each route. Do not send a sensitive prompt to an external classifier to decide whether it was permitted to leave the organization in the first place.

4.2 Redaction, tokenization, and authorized restoration

Combine deterministic recognizers, checksums, domain dictionaries, and entity detection. Test local languages, uncommon names, OCR errors, structured tables, and adversarial spacing. Redact before external processing and before telemetry export. Prefer irreversible removal when the task does not need identity. Use scoped placeholders such as `PERSON_1` when relationships matter, and a protected mapping vault only if later restoration is required.

Placeholders must be request- or conversation-scoped. Two different customers becoming the same string must not create a shared cache hit. Keep a keyed fingerprint of the relevant private bindings in a protected cache namespace, or disable response caching for such content. Restore values only for an authorized recipient at the final release boundary, then validate the restored output. The restoration step itself can introduce leakage that an earlier DLP check could not see.

Store reversible mappings encrypted with strict expiry and access controls. A hash of a low-entropy identifier can be guessed; keyed HMAC reduces that exposure but does not make the underlying data anonymous. Embeddings of confidential content are also confidential artifacts and require access and retention controls.

4.3 Prompt injection: containment is the architecture

Prompt injection can arrive directly from a user or indirectly through retrieved documents, tool output, web pages, OCR, or images. RAG and fine-tuning do not eliminate it [2]. Detection can reduce attack success but cannot prove that an input is safe. Treat the model as a fallible planner operating within a restricted capability set.

Separate trusted policy instructions from untrusted content with structured messages and explicit provenance. Delimiters improve interpretation but are not access control. Do not place secrets in prompts or rely on keeping system instructions confidential. Prevent model-authored text from changing the tool registry, allowed destinations, policy engine, or authenticated identity.

The tool broker validates schemas, applies allowlists, checks current authorization, and constrains destinations independently of the model. Prefer typed business functions such as `lookup_invoice(invoice_id)` over arbitrary SQL or shell execution. For URL tools, control DNS resolution, redirects, protocols, and private or link-local address ranges to reduce server-side request forgery. Use an egress proxy and restrict remote image rendering, which can leak data through crafted URLs.

Threat surface	Primary containment	Detection and residual risk
Poisoned retrieved document	Resource authorization, source provenance, tool isolation	Scan and adversarially evaluate documents; an authorized malicious source can still bias prose.
Tool-response instructions	Typed parser, bounded fields, untrusted-message role	Flag unusual instruction text; never promote it to trusted policy.
Data exfiltration request	Data minimization, destination allowlist, output DLP	Semantic leakage may escape detectors; remove access that the task does not need.
Forged approval or role	Server-side identities and signed approval records	Model text claiming permission has no authority.
Encoded or multimodal payload	Sandboxed parsing and modality-aware testing	No keyword list covers all encodings or languages.

4.4 Toxicity and domain safety

Separate abusive intent from quoting or analyzing harmful material. A support transcript containing offensive language may be legitimate input. Use purpose-specific policies and structured severity labels, evaluate false positives across languages and groups, and offer safe transformations where allowed. A classifier score is a measured signal, not a universal probability or an ethical verdict.

For high-consequence domains, identify the precise disallowed action or unsupported claim. Avoid a single generic toxicity threshold for every route. Record why a decision was made without retaining unnecessary sensitive text. Provide appeal or human-review paths when false positives materially affect users.

4.5 File and schema defenses

Parse PDFs, office documents, archives, images, and HTML in isolated workers with memory, CPU, time, page-count, and expansion limits. Verify file signatures, scan for malware, reject active content where appropriate, and disable parser network access. Bound OCR work; a tiny upload can trigger expensive decompression or image processing. Preserve page and section provenance while removing executable content.

Validate JSON with explicit types, lengths, enumerations, and `additionalProperties: false` where feasible. Reject duplicate keys or ambiguous encodings according to a documented parser policy. Normalize cautiously: preserve meaning, identifiers, and signed content. Check attachment authorization before reading it. Schema validation proves shape, not factual truth, safe intent, or authorization.

Acceptance evidence: redaction recall and false-positive rates are measured on representative sensitive data; test tenants cannot reference each other's attachments; tool and retrieval injections cannot cross privilege boundaries; oversized and malformed files are contained without destabilizing serving capacity.

5. Caching layers and invalidation

5.1 Cache different work at different layers

Exact-match response caching reuses an answer for an equivalent request and execution context. Semantic response caching retrieves a similar prior request and reuses its answer only if a stricter eligibility check passes. Embedding caching reuses vectors for identical content and embedding configuration. Retrieval caching reuses candidate IDs or evidence under a versioned query and access scope. Provider prompt-prefix or KV caching accelerates inference work and may affect billing; it is not the same as returning a prior answer.

Layer	Good candidate	Required scope or version	Main risk
Exact answer	Stable, read-only FAQ	Tenant, entitlement scope, prompt, model, data generation, output schema	Wrong context hidden behind identical text.
Semantic answer	Approved low-risk paraphrases	Exact scope filters plus calibrated semantic match	Similar questions have different entities, dates, negations, or requested actions.
Embedding	Repeated immutable content	Content hash, embedding model, preprocessing, dimensions	Mixed vector spaces or confidential vector exposure.
Retrieval	Repeated query over stable corpus	Query, index generation, filters, entitlement version	Revoked or deleted evidence remains in candidates.
Inference prefix/KV	Repeated model input prefixes	Provider-specific isolation and cache contract	Assuming a provider cache obeys application deletion or tenant rules without verification.

5.2 Key design is part of authorization

An exact key should represent the complete answer-producing context, not merely the question. Include the tenant; a principal or proven equivalent entitlement fingerprint; locale; safe normalized request; relevant conversation-state digest; tool-state version or explicit cache exclusion; model and generation parameters; prompt, policy, schema, and retrieval versions; corpus generation; and private placeholder binding scope when applicable. Use canonical serialization with stable ordering and explicit schema versioning, then a server-keyed digest. Do not expose raw questions in cache keys or metrics.

Sharing across users is permissible only when the result is independent of private state and all recipients have equivalent access. For complex per-resource permissions, use principal-scoped caches until equivalence is demonstrably correct. Do not cache writes, balances, live inventory, permission decisions, or time-sensitive recommendations as generic semantic answers. A content TTL cannot implement immediate entitlement revocation.

Figure 2. Safe cache lifecycle.

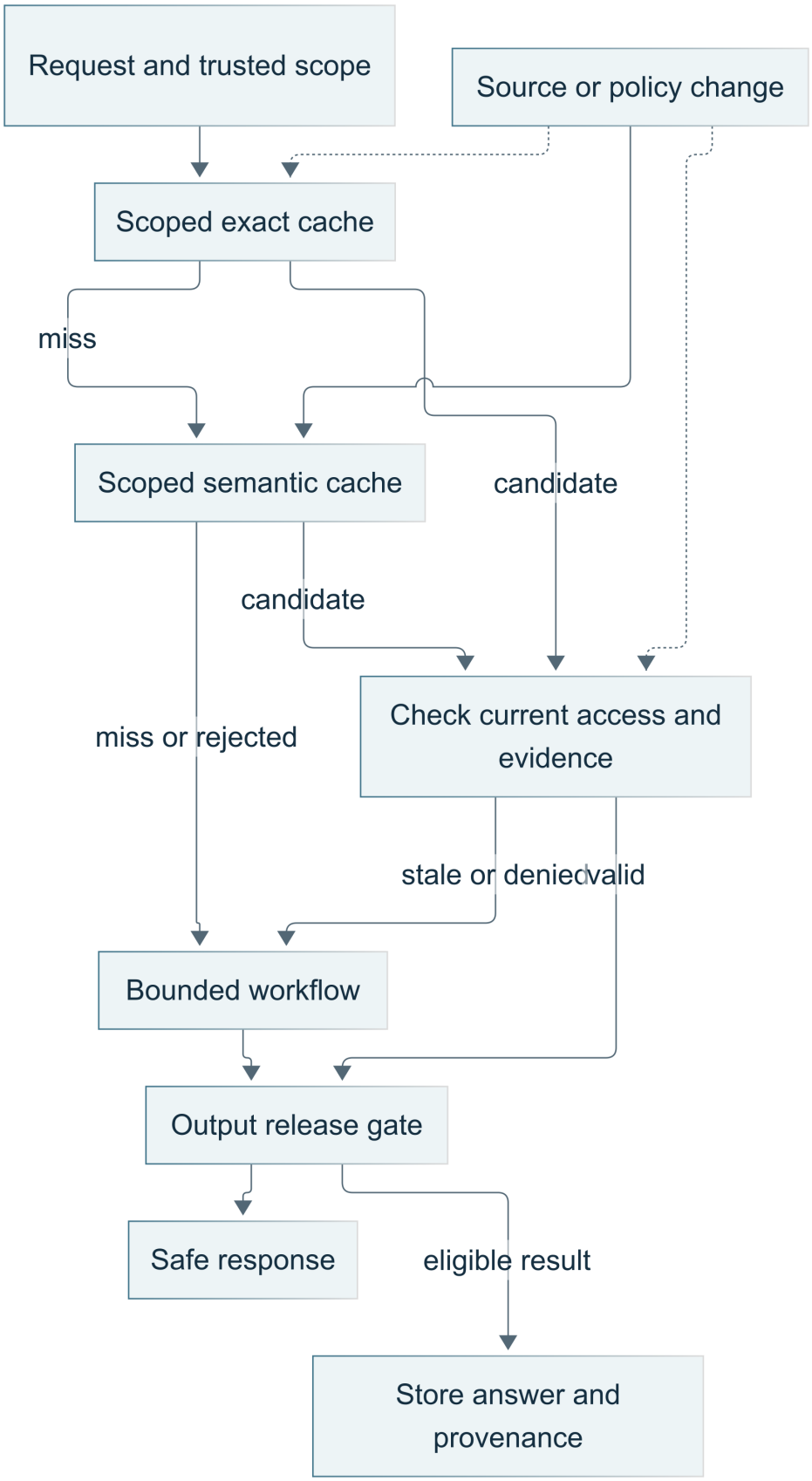


Figure 2. Safe cache lifecycle. Similarity proposes a candidate; scope, provenance, freshness, and current policy decide whether it is usable. Failures at the release gate lead to refusal or review, not to an answer.

5.3 Semantic matching must be selective

Calibrate similarity thresholds on held-out, labeled query pairs for the chosen embedding model and distance function. There is no portable cosine threshold. Include hard negatives: "can" versus "cannot," different fiscal years, currency units, customers, countries, policy editions, and read-versus-write intent. Require exact compatibility for critical entities and temporal scope before comparing semantic similarity. Add a secondary verifier only when its cost is justified; that verifier can also make mistakes.

Measure the precision of accepted cache matches separately from overall hit rate. Cache correctness can be more important than recall: a missed hit costs money, whereas a false hit can disclose data or provide a wrong instruction. Sample hits for independent evaluation and maintain a kill switch for semantic reuse without disabling safer exact or embedding caches.

5.4 Invalidation is a distributed consistency problem

Maintain a reverse dependency index from source document version or resource to cached answers. Every entry records source versions, index generation, entitlement epoch, creation time, expiry, and verification-policy version. Source update events invalidate dependent entries. ACL revocation and deletion publish high-priority events and update an authoritative deny overlay immediately. Read-time checks remain necessary because event delivery and cache eviction can lag.

Namespace versioning provides a reliable coarse fallback: bump a tenant knowledge epoch or policy epoch and stop reading old entries. Fine-grained invalidation improves hit rate but adds bookkeeping. Use both where justified. Protect writes against races: capture versions before generation, then compare them before cache commit and response release. If dependencies changed, discard or regenerate rather than publishing a freshly created stale answer.

Cache TTL bounds tolerated staleness for content, not permission. Stale-while-revalidate may suit public, noncritical material but must not bypass revocation, deletion, or risk policies. Revocation cannot retract content already delivered; define the safety boundary at each release or action and stop future delivery promptly.

5.5 Performance and failure handling

Use request coalescing or a short-lived lease to prevent many misses from generating the same answer at once. Bound waiter time and recover from lease-owner failure. Add randomized TTL jitter to avoid synchronized expiry. Negative-cache only safe, scoped outcomes for short durations; transient permission or dependency failures should not become long-lived refusals. A cache outage should normally fall through to budgeted generation with admission control, not create an unrestricted traffic surge.

For a simplified serial lookup, expected latency is `Llookup + hLhit + (1-h)Lmiss`, where `h` is the accepted-hit fraction. Include embedding lookup, scope checks, output validation, and cache writes in the measured path. A semantic cache that costs more than the model call it avoids can worsen both latency and spend.

Acceptance evidence: replaying identical questions from different tenants never shares private responses; a permission revoke blocks a previously cached answer; a source update racing generation cannot publish an obsolete dependency version; hit precision is measured on adversarial paraphrases.

6. Intent routing, orchestration, and agents

6.1 Route by constraints before complexity

The router chooses an execution plan, not just a model name. First filter plans by data classification, approved region, modality, tool permissions, output contract, context capacity, and risk tier. Then optimize quality, latency, and cost among eligible candidates. A cheap model with the wrong residency or insufficient structured-output reliability is not an eligible fallback.

Start with deterministic route rules for known intents, followed by a small classifier where ambiguity warrants it. Escalate uncertain or high-risk intent to clarification or a controlled path. Keep an intent taxonomy with explicit `unknown` and `out_of_scope` outcomes. Do not use the same uncalibrated model confidence for both correctness and routing.

Request class	Initial execution plan	Escalation or fallback
Deterministic lookup	Authorized database or rule-based response	Ask for missing identifier; do not fabricate data.
Low-risk extraction or classification	Small model with schema validator	One bounded repair or stronger eligible model after observed validation failure.
Knowledge question	Hybrid retrieval, rerank, grounded generation	Query clarification, one retrieval refinement, or abstention if evidence is insufficient.
Complex synthesis	Larger eligible model with selected evidence	Async job when the time budget is incompatible with interactive completion.
Consequential business action	Durable workflow and tool proposal	Policy check and required approval; never a cheaper bypass.

6.2 Model portfolio and fallback semantics

Maintain a model registry with immutable version or deployment identifier, provider, approved data classes and regions, capabilities, tested context limits, quotas, unit cost assumptions, and evaluation evidence. Pin versions when supported; when an alias can change, monitor canary quality and re-evaluate behavior. Treat tokenizer, refusal behavior, tool calling, and output schemas as compatibility concerns.

Classify failures before retrying. Retry transient transport errors or rate limits only within the remaining deadline and quota, with jitter and provider retry guidance. Do not retry authentication errors, deterministic schema rejection by the provider, or policy refusals as if they were network faults. Route quality failures through a bounded escalation policy. Never send the same rejected request to progressively less restrictive providers until it succeeds.

Use circuit breakers by provider, deployment, and region; preserve a small controlled probe rate. Bulkheads separate pools so a slow tool or tenant cannot exhaust all workers. A fallback adds latency and may increase data exposure, so re-run eligibility and record the reason. Once user-visible streaming begins, do not silently splice in another model's continuation. End the stream with a clear terminal status or restart a labeled attempt under an explicit client protocol.

6.3 Prefer explicit workflows for predictable tasks

A workflow can be a finite-state machine: validate, retrieve, generate, check, release. It is easier to test, budget, and audit than an open-ended agent. Introduce model-chosen steps only when fixed logic cannot reasonably handle the task and evaluation demonstrates a benefit. Parallelize independent read-only retrievals when helpful, but account for total token spend, shared quotas, and correlated failures.

For agents, set maximum steps, wall time, total tokens, cost, tool invocations, and repeated-action limits. Detect loops with normalized action signatures. Track a typed state object containing task objective, facts, provenance, remaining budget, and pending approvals. Treat summaries as lossy model output: preserve authoritative identifiers, approvals, and policy state separately from conversational summaries.

Figure 3. Durable agent lifecycle.

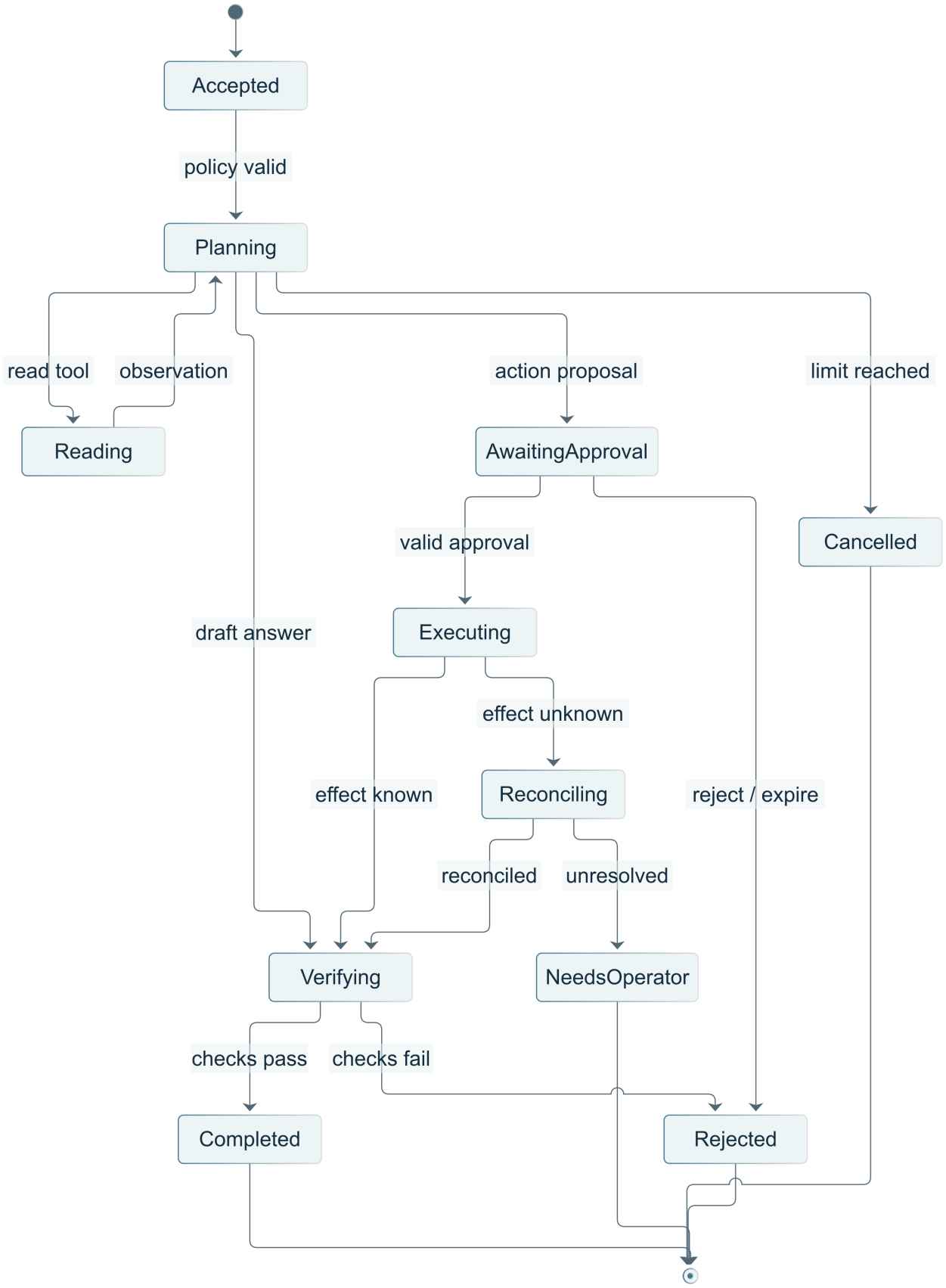


Figure 3. Durable agent lifecycle. A tool timeout may leave an uncertain effect; reconciliation prevents blind re-execution. Cancellation and recovery controls also apply to long-running tool states.

6.4 Durable state and side effects

Persist state transitions and outbox events in one transaction. Workers claim tasks with leases and fencing tokens so an expired worker cannot commit over a new owner. Read-only calls can often be retried safely; mutating calls need business-level idempotency or authoritative reconciliation. Do not claim exactly-once execution across arbitrary external APIs. A timeout can mean that the external action succeeded and the reply was lost.

Bind approvals to a canonical action digest containing tenant, principal, tool, resource, arguments, expected resource version, maximum amount where relevant, and expiry. Before execution, recheck authorization, approval validity, and resource preconditions. If a model changes the action, obtain a new approval. Compensation is a separate authorized action, not an automatic reversal that is always possible.

6.5 Memory is a governed data store

Separate short-lived conversation context from persistent user preferences and business records. Persist only facts with an explicit purpose, retention rule, and provenance. Give users appropriate controls to inspect or delete memory. Do not let one conversation inject tenant-wide instructions or authoritative facts. Reauthorize retrieved memory, include relevant memory versions in cache eligibility, and avoid storing secrets in model-generated summaries.

Acceptance evidence: all routes honor residency and data policy during failover; step and cost caps stop loops; worker restarts preserve jobs; a lost tool response cannot produce a duplicate effect; approval cannot be replayed with modified arguments.

7. RAG ingestion and data pipelines

7.1 Begin with ownership and source contracts

A vector database cannot fix an unowned knowledge base. Register each source with a business owner, canonical identifier, classification, allowed tenants, access-control semantics, expected update frequency, deletion policy, and retention rules. Distinguish authoritative policy documents from informal commentary. Resolve duplicate sources and conflicting versions before treating retrieval results as evidence.

Use connectors with the least privilege required to read approved content and permissions. Store raw source snapshots in encrypted object storage with content hashes and immutable version identifiers. Keep raw files separate from normalized text, chunks, embeddings, and indexes because they have different access, retention, and reprocessing needs. Preserve a lineage manifest linking every derivative to source version and transformation versions.

7.2 A replayable ingestion pipeline

Figure 4. RAG data pipeline.

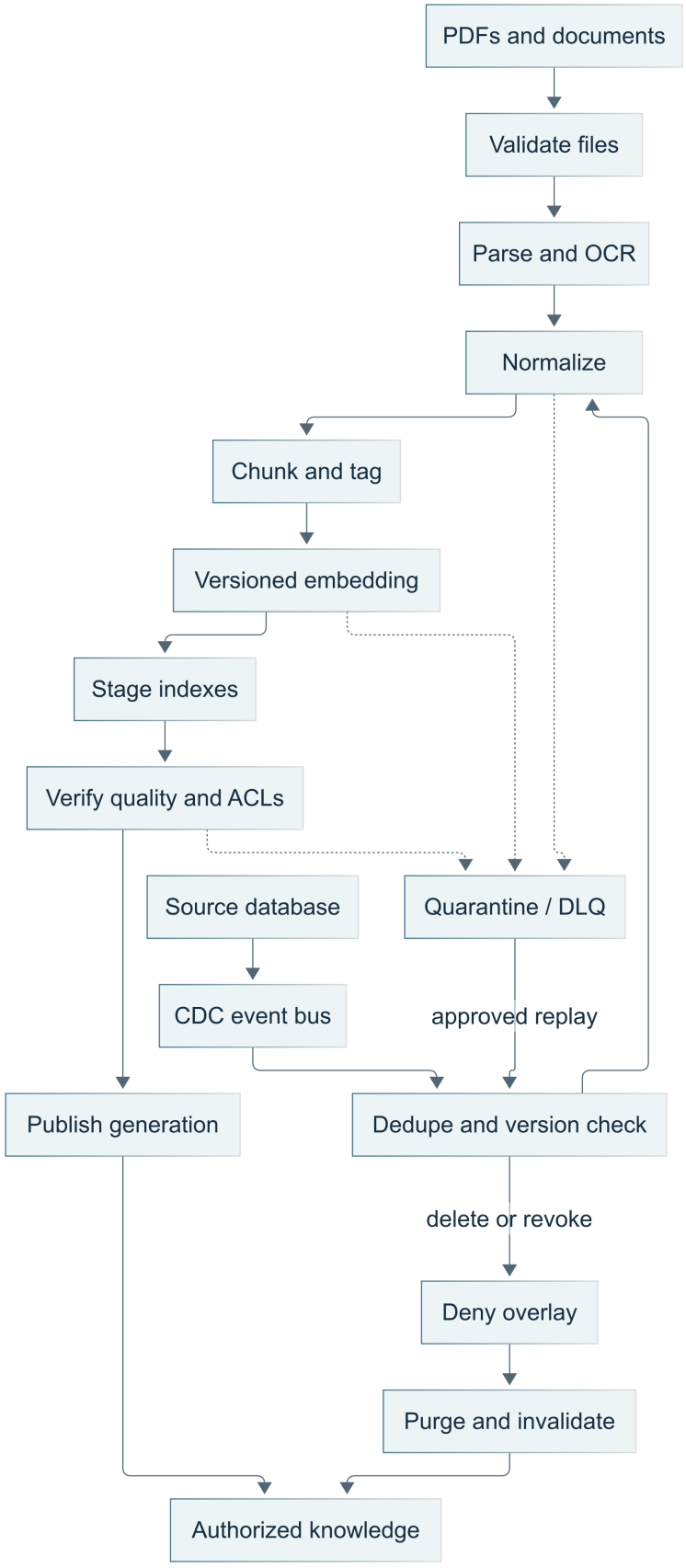


Figure 4. RAG data pipeline. Query eligibility is controlled by an active manifest, not by whichever index write happens to finish first. The immediate deny overlay prevents asynchronous cleanup from becoming an access-control gap.

Each stage must be idempotent and restartable. Use a work key such as `(tenant, source_id, source_version, stage, transform_version)`. Store status, attempts, leases, and error categories. Checkpoints acknowledge durable outcomes, not merely receipt of a message. Dead-letter queues need owners, dashboards, replay controls, and retention; otherwise they become silent data loss.

7.3 Parsing PDFs and documents without destroying meaning

Preserve headings, page numbers, section paths, tables, list hierarchy, footnotes, and document order. Multi-column PDFs and scanned tables often fail naive text extraction. Use OCR only when required, record extraction confidence, and quarantine poor extraction rather than confidently indexing nonsense. Keep both normalized text and coordinates or offsets that support accurate citations.

For tables, retain column headings and units with each row group. For code, chunk around semantic units and retain file path and language. For long policies, preserve definitions and exceptions near the governed clauses. Add document title and section context to retrieval representations while retaining the original excerpt for evidence. Avoid using generated summaries as the sole authoritative source unless independently verified.

7.4 Chunking is an evaluated design choice

Begin with structure-aware chunks and modest overlap, then tune on retrieval recall and answer quality. An illustrative experiment might compare 300, 600, and 1,000 tokens with 10-15% overlap; these are candidate settings, not universal defaults. Smaller chunks improve specificity but can lose context. Larger chunks preserve meaning but dilute embeddings and consume prompt capacity. Excessive overlap duplicates evidence, increases storage, and may distort ranking.

Use parent-child retrieval where useful: search small child chunks, then expand to a bounded parent section after authorization. A chunk should include source ID, version, page or section, text hash, language, classification, permission reference, valid time, parser version, and chunker version. Distinguish source event time, ingestion time, and query-visible publication time.

7.5 CDC ordering, snapshots, and tombstones

Change Data Capture reads database changes and feeds downstream consumers; Debezium documents a log-based architecture and supported connector patterns [4]. CDC does not automatically provide atomic updates across a vector index, lexical index, object store, and cache. Design for duplicate delivery, out-of-order processing, connector restarts, and schema evolution.

Partition events by stable source entity so related updates are ordered where the transport supports it. Carry an event ID and a source-native version or position with a documented comparison rule; timestamps alone can be ambiguous. A consumer applies an event only if it is newer than the last applied version for that entity. Deduplicate within the same transaction that changes metadata or records the next outbox job.

For bootstrap, take a consistent snapshot with a connector-supported boundary and then continue from the corresponding log position. Reconcile counts and hashes periodically because some source systems have incomplete webhook or delete semantics. Monitor source-log retention and connector lag; an expired replication position may require a new snapshot rather than a blind restart.

Deletes need tombstones that outlive possible replay windows. A delayed old update must not resurrect a deleted document. Permission changes may arrive independently from content updates; version and prioritize them independently. Update the authoritative authorization or deny service before relying on bulk index cleanup. Propagate deletes to raw objects where permitted, parsed text, vectors, lexical indexes, caches, review copies, exports, and retained evaluation datasets.

7.6 Publish coherent generations

Stage new derivatives under a generation ID. Verify embedding dimensions, model version, record counts, required metadata, ACL coverage, and representative retrieval tests. Publish a manifest pointing to the new coherent generation only when required stores are ready. Readers capture a generation and use it consistently for the request. If cross-store atomic cutover is unavailable, use a central manifest plus per-store readiness checks and return a controlled degraded result when required consistency is unavailable.

Re-embedding changes the coordinate system: never mix vectors from incompatible embedding models in one search space. Build a parallel index, backfill, shadow-query both versions, compare recall and latency, switch the manifest, and retain rollback capability within retention policy. Changes to chunking, parsing, or metadata may require the same discipline even if embedding dimensions stay identical.

Pipeline pattern	Appropriate use	Operational trade-off
Scheduled batch	Small, slowly changing corpus	Simple operations; freshness bounded by schedule and job duration.
Webhook/event ingestion	Sources with reliable change notifications	Low latency; requires deduplication, reconciliation, and delete handling.
Log-based CDC	Database-backed knowledge with frequent updates	Rich change stream; connector privileges, log retention, and schema changes need ownership.
Full generation rebuild	Major parser, chunker, or embedding migration	Clean rollback and reproducibility; temporary double storage and compute.
Incremental generation updates	Large corpus with localized changes	Efficient; more complex manifests, dependency tracking, and consistency rules.

7.7 Storage sizing and freshness budgets

Raw float32 vector storage is approximately `chunk_count dimensions 4 bytes`. Five million 1,536-dimensional chunks require about 30.72 GB of raw vector values in decimal units. This excludes graph indexes, metadata, original text, replicas, write amplification, and backups. Measure real index overhead and filtered-query performance with a representative corpus before selecting infrastructure.

Split freshness latency into source notification, queue wait, extraction, embedding, index write, and publication. Apply backpressure when embedding quotas or index writes lag. Reserve capacity for deletes and ACL changes so a large backfill cannot starve revocation. A freshness SLO should expose the oldest unapplied change as well as average lag; the average can hide a stuck partition.

Acceptance evidence: duplicate and reordered events converge to the correct version; delayed updates cannot resurrect deletes; a source disconnect is observable; mixed embedding generations are rejected; a new index can be rolled back without undoing revocation.

8. Retrieval and grounded answer construction

8.1 Hybrid search and permission filtering

Lexical search matches exact names, codes, dates, and rare terms. Dense vector search captures semantic similarity. Hybrid retrieval combines both, often with reciprocal rank fusion (RRF) to avoid directly comparing incompatible score scales. A common formula is `score(d) = sum_i 1 / (k + rank_i(d))`, with one-based ranks and a chosen smoothing constant `k`; tune candidate sizes and parameters on your corpus. Elasticsearch documents an RRF implementation [6].

Apply tenant and resource constraints in the search engine where possible, then perform an authoritative check before using returned evidence. If an engine applies filtering after approximate nearest-neighbor selection, highly selective permissions can reduce recall. Over-fetching can improve candidates but does not justify sending unauthorized text to a reranker or model. Fetch IDs first, authorize, then load text when the storage architecture requires a separate entitlement service.

Figure 5. Query-time RAG.

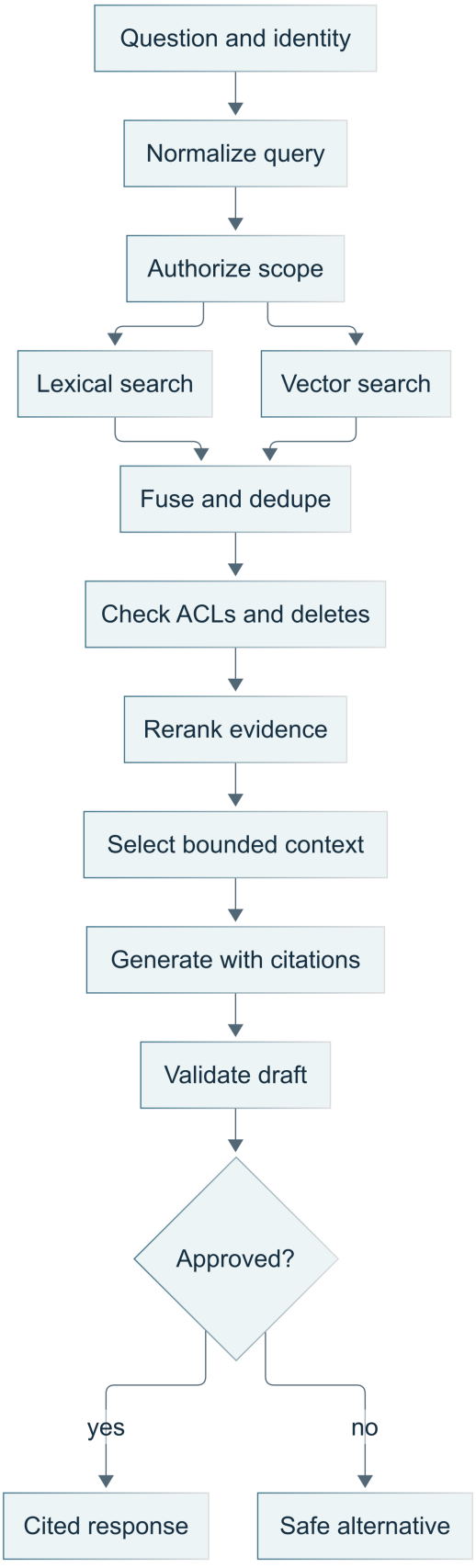


Figure 5. Query-time RAG. The evidence selection step balances relevance, coverage, diversity, and token budget. Citation IDs resolve only to authorized source versions.

8.2 Choosing a vector and search platform

Option	Favor when	Evaluate carefully
Relational store with vector extension	Existing SQL operations, joins, transactions, and moderate search needs	ANN index memory, maintenance, filtered recall, replication, and isolation.
Dedicated vector database	Large vector workloads or specialized search features	Metadata filtering semantics, durability, backup/restore, tenant isolation, and consistency.
Search engine with lexical and vector retrieval	Rich keyword search, facets, and hybrid relevance	Ranking configuration, resource contention, permission filtering, and operating cost.
Managed retrieval service	Reduced operational ownership is valuable	Residency, deletion behavior, opaque ranking, portability, and export/rebuild options.

Benchmark candidate platforms with the same query set, corpus, permission distribution, and concurrency. Compare recall at fixed latency and cost, not only advertised scale. HNSW commonly trades memory and construction cost for query performance; IVF-style approaches depend on representative training and probe choices. Exact search is useful as a correctness baseline and may be adequate for small partitions. pgvector's documentation describes exact and approximate search and the effect of filtering on approximate queries [5].

8.3 Query rewriting and reranking

Rewrite ambiguous conversational references using authorized conversation state, but preserve names, identifiers, negation, units, and time. Search the original query alongside the rewrite when needed. Log rewrite version and evaluate whether it changes user intent. Multi-query expansion can improve recall while increasing retrieval cost and injection surface; cap the number of expansions.

Rerank a bounded candidate set with a cross-encoder or eligible model. Select evidence for coverage and diversity, not just near-duplicate high scores. For conflicting sources, prefer explicit authority and effective-date rules; present the conflict or abstain when it cannot be resolved. A relevance score indicates match quality, not truth. Treat retrieved instructions as source content, never as a replacement for application policy.

8.4 Context packing and citation fidelity

Budget context as `system + question + history + tools + evidence + reserved_output + margin <= supported_context`. Count with the actual model tokenizer where available. Reserve output capacity before filling evidence. Use bounded source excerpts with stable citation IDs, titles, versions, and page or section references. Preserve exceptions and definitions that qualify a claim. Put redundant or low-value text outside the prompt rather than assuming a larger window makes every token useful.

Require the generated answer to distinguish supported facts, uncertainty, and proposed next steps. Validate that citation IDs exist in the supplied evidence and that cited passages support the associated claims. A valid URL or matching document title is not support. Resolve download links through an authorization gateway with short-lived access rather than exposing unrestricted object-store URLs.

8.5 Abstention and retrieval evaluation

When authorized evidence is absent, stale, contradictory, or insufficient, ask a targeted clarification or return an explicit limitation. Do not substitute general model memory for a question that requires enterprise-specific facts. For answerable queries, evaluate recall@k, precision@k, mean reciprocal rank, and nDCG as appropriate. For unanswerable queries, measure false-answer and appropriate-abstention rates.

Ground-truth evidence sets can be incomplete. Document annotation rules, allow multiple valid passages, and review apparent false positives. Slice metrics by language, document type, permission density, source age, tenant size, and query complexity. Include negative-permission queries: excellent semantic relevance with unauthorized evidence is a security failure, not successful retrieval.

Acceptance evidence: retrieval finds the correct authorized source for hard exact identifiers and paraphrases; citations map to real supporting passages; empty or contradictory evidence triggers the intended behavior; selective ACLs do not silently destroy recall.

9. Output guardrails and safe delivery

9.1 Evaluate independent dimensions

Output validation checks structure, support, privacy, policy, and executable consequences. These dimensions are separate: a perfectly valid JSON object can contain false facts or unauthorized data. Use deterministic checks where possible and probabilistic checks where necessary. Record which checks ran, their versions, and any degraded mode.

Check	Implementation pattern	Limitation and response
Schema and formatting	Parser, JSON Schema, enum and length checks	Shape is not meaning; one bounded repair then reject or review.
Citation integrity	Resolve IDs, versions, pages, and access	Existing citation does not prove support; inspect claim-to-passage alignment.
Factual support	Claim extraction plus evidence comparison; deterministic tools for arithmetic	Model judges can share errors; abstain or review material uncertainty.
Leakage prevention	Secret patterns, DLP, canaries, destination checks	Semantic disclosure may evade patterns; minimize data available to generation.
Domain policy	Versioned rules, content classifiers, risk routing	Policy interpretation can be ambiguous; route high-consequence cases to qualified review.
Executable output	Typed commands, parameterized operations, sandboxing	Never execute prose, SQL, HTML, or shell merely because the model produced it.

9.2 Hallucination checking is evidence assessment

Break important claims into checkable units: amounts, dates, policy conditions, entity relationships, and causal assertions. Verify arithmetic with code and factual claims against authoritative evidence. Use an independent verifier or judge for semantic entailment, calibrated against human labels. Do not report the model's self-declared confidence as a probability of correctness. Multiple agreeing models may share the same blind spot.

Measure groundedness and factual correctness separately. A statement can accurately reflect a stale document yet be wrong today. A fact can be true but unsupported by the evidence supplied to this request. For high-risk answers, require a current authoritative source and human review when essential evidence is missing. No general-purpose hallucination detector offers a complete correctness guarantee.

9.3 Leakage and downstream security

Run output DLP before release, including after placeholder restoration or tool-result formatting. Scope what a user can receive independently from what a service account can technically access. Detect credential-like strings, confidential identifiers, unexpected bulk extraction, and attempts to encode data for export. Apply maximum result sizes and purpose-specific field selection at the tool or data layer to reduce exposure earlier.

Render text safely. Sanitize allowed Markdown and HTML, escape untrusted content in the destination context, disallow executable scripts, and restrict externally loaded resources. Use parameterized queries and typed API clients for structured results. An output validator must not itself fetch model-provided URLs with unrestricted network access. Log safe reason codes rather than rejected secrets.

9.4 Streaming is a release-policy decision

For high-risk content, buffer the complete draft and release only after validation. Users may receive progress events that contain no unchecked answer text. For lower-risk chat, incremental moderation can check bounded chunks with overlap to reduce boundary splitting, but later context can still reveal that an earlier chunk was unsafe or wrong. Clearly distinguish draft text from final verified content when that product behavior is acceptable.

Delivery mode	Benefit	Cost and safety boundary
Full buffering	Complete response can be validated before release	Longer time to first answer; best for sensitive or consequential content.
Sentence or segment buffering	Some responsiveness with local checks	Cross-segment context remains incomplete; emitted text cannot be recalled.
Immediate token streaming	Lowest apparent generation delay	Cannot guarantee pre-release full-answer validation; restrict to suitable risk tiers.

If validation fails, allow only a bounded correction using explicit error feedback and the same authorization scope. Revalidate the entire repaired output. Repeated repair loops can amplify cost and produce hidden policy bypasses. Fall back to a deterministic refusal, clarification, or review receipt after the allowed attempt count.

9.5 Compliance controls need a policy owner

Translate applicable organizational and jurisdictional requirements into owned, testable controls: data residency, retention, consent or other processing basis where relevant, access restrictions, explanation requirements, and review obligations. Maintain a policy version and change history. An LLM compliance check is one signal; it does not establish legal compliance. Have the relevant policy or legal owner approve the mapping for the actual use case and geography rather than treating this handbook as legal advice.

Acceptance evidence: invalid schema never reaches consumers; fabricated citations are rejected; sensitive restoration is rechecked; high-risk streams release no unchecked text; repairs remain within the original budget and policy.

10. Human-in-the-loop operations

10.1 Escalate for a reason that a reviewer can act on

Triggers include consequential actions, missing or conflicting authoritative evidence, suspected leakage, unresolved identity or permission anomalies, repeated validation failure, ambiguous policy, low retrieval coverage, and explicit user requests. Calibrate confidence-related triggers with labeled outcomes. A low model score without an actionable reason creates review noise and may miss confidently wrong answers.

Distinguish **approval before action**, **review before answer release**, and **post-release quality sampling**. Only the first two prevent the pending action or answer from being released. Post-release review supports improvement and incident discovery but cannot undo already disclosed information.

Review pattern	Appropriate use	Trade-off
Interactive clarification	Missing intent or an identifier the user can supply	Fast and contextual; the user is not necessarily an authorized approver.
Asynchronous approval or review	Consequential effects or material uncertainty	Strong release control; requires staffing, expiry, and durable state.
Post-release sampling	Low-risk quality monitoring and dataset improvement	Scales more cheaply; cannot prevent the already released outcome.

10.2 Durable review queues

Create a review case transactionally with the paused workflow state. Include tenant, requesting principal, reason, risk tier, sanitized draft, evidence references and versions, proposed action digest, policy version, expiry, and required reviewer role. Store sensitive evidence in a protected evidence vault rather than duplicating it into queue payloads or email. Authorize every reviewer access and prevent cross-tenant search leakage.

Return `202` with a case or job reference where asynchronous behavior is appropriate. Provide authenticated status retrieval and optional scoped webhook callbacks. Sign callbacks, retry with bounded backoff, and deduplicate delivery. Communicate pending, approved, rejected, expired, and needs-more-information outcomes explicitly. Do not keep an interactive HTTP request open for a human decision that may take hours.

Figure 6. Review is a governed state transition.

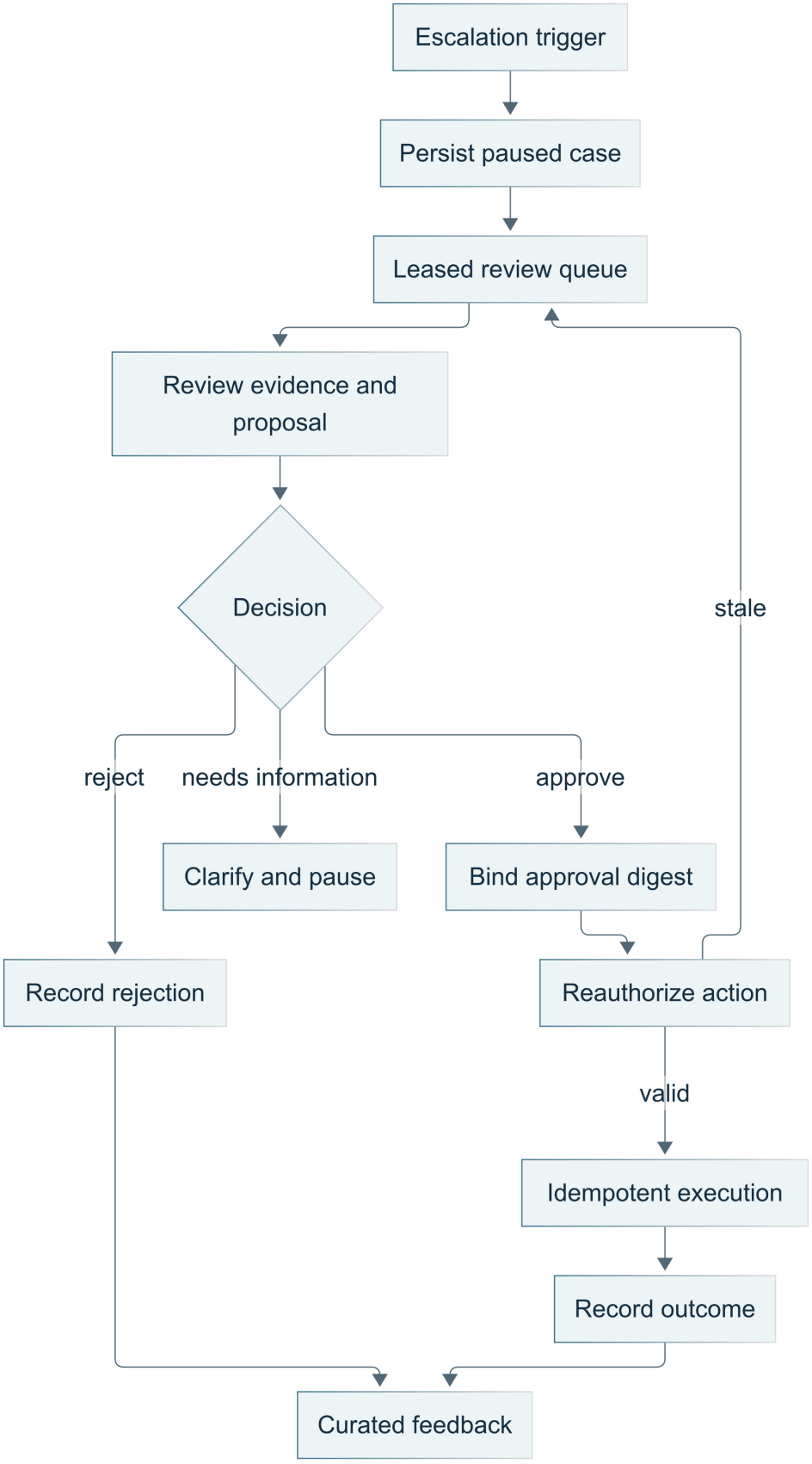


Figure 6. Review is a governed state transition. Reviewer edits that change action semantics create a new proposal; any required additional approval applies to that new proposal.

10.3 Reviewer experience and separation of duties

Show the user's requested outcome, the exact proposed effect, supporting sources, unresolved conflicts, and a concise rationale summary. Do not require hidden model reasoning or present verbose generated explanations as evidence. Make the safe action easy: approve, reject, edit within permitted bounds, request information, or escalate. Flag stale evidence and expired approvals visibly.

Use separation of duties for high-impact actions: the requester should not automatically be the approver, and particularly sensitive operations may require two independent reviewers. Record decision identity, timestamp, reviewed content digest, and reason. An approval click without a bound payload is insufficient. Limit reviewer privileges to the cases and actions assigned to them.

10.4 Capacity and feedback quality

Estimate review demand before launch. At 10,000 requests/hour with 2% escalation, arrivals are 200 cases/hour. If each review takes 3 minutes, one reviewer handles 20 cases/hour at full utilization. Ten reviewers would be continuously busy; at a planned 70% utilization, at least `ceil(200 / (20*0.7)) = 15` concurrently staffed reviewers are needed before allowances for breaks, shifts, and specialist routing. Variability and bursts can require more. Track oldest-case age, percentile wait time, abandonment, and SLA breaches.

Define severity-based response targets and an expiry behavior. If the review system is unavailable or overloaded, hold or reject actions requiring approval; do not auto-approve. For low-risk answers, use a preapproved fallback if one exists. Page the owning team when urgent cases approach their deadline.

Treat feedback as untrusted data. Separate user satisfaction from factual correctness; collect reason codes and verified corrections. Sample routine successes as well as failures to avoid biased datasets. Have curators remove secrets, resolve label conflicts, and enforce allowed training use before adding examples to evaluation or fine-tuning sets. Version datasets and track reviewer agreement. A thumbs-up should never automatically rewrite a system policy.

Acceptance evidence: review records survive worker restarts; approval expires and cannot authorize altered arguments; queue capacity meets expected demand; reviewer changes are auditable; the system has a safe behavior when nobody is available to approve.

11. Load balancing, capacity, and cost

11.1 Model traffic is shaped by tokens and time

HTTP request rate alone understates inference demand. Track input-token rate, output-token rate, active sequences, context lengths, and time held per request. A provider may enforce request and token quotas independently. Long-context prefill and output decoding stress different resources. For self-hosted models, KV-cache memory can be the limiting factor even when GPU compute is not fully utilized.

Route to eligible deployments using queue depth, outstanding token estimates, recent tail latency, error rates, and quota headroom. Least-connections can help with long-lived streams but ignores differences in sequence size. Weighted token-aware load balancing is often more informative. Preserve tenant fairness and reserve capacity for critical routes; do not let batch embeddings exhaust interactive inference capacity.

11.2 Worked capacity example

For the illustrative burst of 200 requests/second and a 40% approved response-cache hit rate, generation demand is `200 * 0.60 = 120` requests/second. At 3,000 input and 500 output tokens per generation, demand is 360,000 input tokens/second and 60,000 output tokens/second, before retries, judges, embeddings, or tool planning. That is 21.6 million input and 3.6 million output tokens/minute. Capacity agreements must cover both throughput and request limits.

If mean generation service time is 4 seconds, Little's Law gives approximately `L = lambda W = 120 4 = 480` in-flight generations in a stable system. This is a concurrency estimate, not a tail-latency guarantee and not a GPU count. Validate the service-time assumption under actual prompt lengths and concurrency. With a hypothetical sustainable 20 generations/second per replica at the tested mix and a planning utilization of 60%, a first estimate is `ceil(120 / (20*0.60)) = 10` replicas. Then test burst, cold-cache, and one-zone-loss scenarios.

At zero response-cache hits, the same burst requires 200 generations/second. Design a documented cold-cache mode: additional capacity, conservative admission, shorter optional output, or a controlled queue. Cache warm-up is not a substitute for capacity planning. Do not use average daily QPS, approximately 11.6 for 1 million requests/day, to size a 200 QPS burst.

11.3 Deadlines and latency budgets

Assign one end-to-end deadline and pass the remaining budget downstream. An illustrative 12-second buffered route might allocate 0.3 seconds to edge and input checks, 0.1 to cache, 1.2 to retrieval and reranking, 8.5 to generation, 1.2 to output checks, and 0.7 to scheduling/network margin. These allocations sum to 12 seconds but are not additive p95 predictions. Measure the full latency distribution and correlated slowdowns.

First-token latency includes admission wait, retrieval, provider queueing, and prefill. Total completion latency also depends on output length and decode speed. Streaming improves perceived responsiveness but does not necessarily reduce completion time or cost. Hedged requests can reduce tails at the expense of duplicated work; use only for safe idempotent reads, cancel losers, and count all incurred usage. Avoid hedging expensive model calls by default.

11.4 Token and monetary budgets

Reserve a worst-allowed spend before execution, reduce it as attempts consume budget, and settle actual provider usage. Maintain separate accounting for uncached input, provider-cached input, output, reasoning tokens when exposed and billed, embeddings, reranking, evaluation, tools, and self-hosted compute. Provider totals may include categories differently; do not double-count cached tokens or infer missing usage as zero.

For one model attempt, a simplified cost is `CuTu/1e6 + CcTc/1e6 + Co*To/1e6`, where `Tu` is uncached input, `Tc` cached input, and `To` output, and prices are per million tokens. Add every attempt, including failed or cancelled attempts that incurred charges. Reconcile with provider billing exports because local streaming counters may be incomplete.

Illustrative economics, not vendor pricing: assume input is $1/million tokens and output is $4/million. A 3,000-input, 500-output call costs `$0.003 + $0.002 = $0.005`. At 1 million daily requests and a 40% response-cache hit rate, 600,000 calls cost $3,000/day before supporting services. If 10% of those calls incur one additional equivalent attempt, generation becomes $3,300/day. This excludes cache operation, retrieval, judges, storage, networking, human review, and staff. Optimize total cost per successful task, not only cost per model call.

11.5 Horizontal scaling and self-hosted inference

Keep API workers stateless where possible; move durable job state, conversation data, idempotency, and quota reservations to shared stores. Scale from queue wait and token demand as well as CPU. CPU alone is a poor signal for workers awaiting remote inference. Use readiness gates for model loading, graceful draining for streams, bounded termination windows, minimum warm capacity, and zone-aware placement.

For self-hosting, benchmark continuous batching, quantization, parallelism, and KV-cache policies on the actual workload. Batching can increase throughput while worsening latency for short requests. Quantization can reduce memory and cost but needs quality and safety regression tests. Model licenses, image provenance, GPU driver/runtime compatibility, patching, and supply-chain integrity become operating responsibilities. Managed APIs reduce these duties but introduce quota, contract, and provider-dependency constraints.

Optimization	Expected benefit	Guardrail or trade-off
Smaller model for eligible intents	Lower latency and token cost	Route-specific quality evaluation; escalation must remain policy-compatible.
Shorter evidence and output	Less prefill and decoding	Preserve required facts, exceptions, citations, and user intent.
Async batching	Higher utilization for noninteractive work	Explicit completion SLA and isolation from interactive capacity.
Response and embedding caching	Avoid repeated work	Scope, freshness, version, and permission correctness.
Continuous GPU batching	Higher aggregate throughput	Test tail latency, sequence fairness, and memory exhaustion.
Multi-provider fallback	Resilience and quota diversification	Extra evaluation, data-governance review, and correlated outage analysis.

Acceptance evidence: burst and cold-cache demand meet declared behavior; cost reservations prevent concurrent overspend within defined bounds; all attempts appear in usage; draining preserves or safely terminates streams; scaling remains effective during a zone loss.

12. Monitoring, observability, and evaluation

12.1 Make the execution path explainable

Create a root trace for the user operation and child spans for admission, input checks, cache lookup, route selection, retrieval, reranking, each model attempt, tools, review handoff, and output validation. Propagate trace context through queues and connect delayed jobs with trace links where appropriate. Keep a separate immutable audit trail for privileged decisions; sampled tracing cannot be the only action ledger.

Record request/job IDs, pseudonymous tenant reference, route, risk tier, model deployment, prompt and policy versions, index generation, cache status, retry/fallback reason, attempt number, token usage, and terminal outcome. Use bounded-cardinality dimensions for metrics. Put request IDs and evidence references in traces or logs, not metric labels. Avoid exposing raw prompts, secrets, full retrieved documents, or hidden model reasoning in default telemetry.

OpenTelemetry defines GenAI semantic conventions, but teams should pin the conventions and instrumentation version and review maturity and migration notes before adopting attributes [7]. Keep an internal mapping layer so vendor SDK changes do not silently break dashboards. Treat missing provider usage as unknown and reconcile later rather than recording zero.

12.2 A metric catalog with owners

Dimension	Essential measurements	Typical owner and action
Reliability	Success by outcome, timeouts, provider errors, circuit state	SRE: investigate burn rate and dependency failures.
Latency	Queue wait, first released token, total time, per-stage p50/p95/p99	Platform: locate critical path and saturation.
Usage and cost	Input/output tokens, all attempts, reserved versus actual spend, cost/success	Platform and FinOps: route or budget changes after quality checks.
Retrieval	Empty evidence, recall on labeled sets, filter rejection, reranker latency	Search team: corpus, ranking, or permission investigation.
Freshness	Commit-to-visible lag, oldest event, partition lag, failed source sync	Data platform: repair connector or replay partition.
Safety	Policy blocks, DLP findings, injection test success, unauthorized action attempts	Security: contain and inspect boundary failures.
Quality	Correctness, groundedness, citation support, task completion, abstention	Applied AI and product: dataset and pipeline improvements.
Human review	Arrival rate, oldest case, review time, disagreement, approval expiry	Operations: staffing, policy, and escalation adjustment.

Dashboards should show cohorts by route and release version. Global averages can hide a language regression or one tenant with unusually restrictive permissions. Distinguish cache-hit latency from

generated-answer latency and provider first token from user-visible first validated token. Expose telemetry sampling rates and dropped-event counts so operators know what they cannot see.

12.3 Select tracing and evaluation tools by constraints

LangSmith supports dataset-based offline evaluation and evaluation of production interactions [8]. Arize Phoenix provides tracing, evaluation, datasets, and experiments, and accepts OpenTelemetry-based traces [10]. These are examples, not required dependencies or claims of feature parity. Evaluate data residency, private deployment options, access controls, retention, exportability, instrumentation overhead, and licensing against current product terms.

Use one stable run ID across the application, observability backend, evaluation results, and audit records. Export sanitized traces asynchronously with bounded buffers. A monitoring outage should not normally take down answers, but operations requiring durable audit may need to pause if their audit record cannot be committed. Define this distinction explicitly.

12.4 Offline, online, and human evaluation

Build a versioned evaluation corpus from representative permitted examples, curated edge cases, historical failures, and adversarial cases. Keep development and held-out release sets separate; split by source or time where necessary to reduce leakage. Include answerable, unanswerable, ambiguous, stale, multilingual, permission-denied, and injected queries. Document annotation guidelines and source versions.

Use deterministic assertions for schema, access, arithmetic, and tool effects. Use human labels for material semantic quality and calibrate LLM judges against them. Prefer explicit rubrics and bounded categories over vague 1-10 scores. Randomize answer order in pairwise evaluation to assess position bias, conceal candidate identity where feasible, and monitor judge drift. Retrieved malicious content can attack the judge too; isolate evaluator inputs and capabilities.

Automated RAG or agent evaluation frameworks can accelerate scoring, but metric names do not establish validity. Check what context a metric sees, its denominator, and whether it uses a model judge. An answer-faithfulness score is not an end-to-end business-success score. For agents, verify final state and prohibited side effects as well as the output text.

Run online evaluators on a privacy-approved sample, oversampling errors and high-risk routes while retaining an unbiased baseline sample. Weight estimates appropriately when reporting population rates from stratified sampling. Online findings should create curated regression cases, not automatically deploy new prompts or models.

12.5 Statistical and operational release discipline

Report sample size, confidence intervals, and important slices. Compare candidates on the same examples and use paired analysis where appropriate. Repeated model runs reveal variability; temperature zero does not guarantee identical execution across provider changes. A zero-failure test is not proof of zero risk. With 1,000 independent representative trials and no failures, the rough rule-of-three upper 95% failure bound is about 0.3%; adversarial tests are not usually independent representative trials, so this bound does not characterize all attacks.

Freeze rollout if a critical safety invariant fails regardless of average quality. For noncritical quality metrics, predeclare acceptable regressions and improvement targets. Alert on sustained multi-window error-budget burn rather than every isolated slow response. Also use immediate alerts for confirmed cross-tenant leakage, unauthorized actions, and audit loss on consequential workflows.

Acceptance evidence: one request can be reconstructed without raw-secret logs; a prompt or index regression is attributable to a release; evaluators agree sufficiently with reviewers for their intended use; spend and provider bills reconcile within documented tolerances.

13. Data model, contracts, and state

13.1 Authoritative state versus derived state

Store tenant membership, permissions, job state, approvals, action records, and idempotency in authoritative transactional systems. Treat vectors, lexical indexes, caches, and analytics as derived stores. Derived stores must be rebuildable from retained authoritative sources and transformation versions. A stale search result can be discarded; a lost approval or uncertain payment effect needs a stronger recovery procedure.

Figure 7. Knowledge and access ER model.

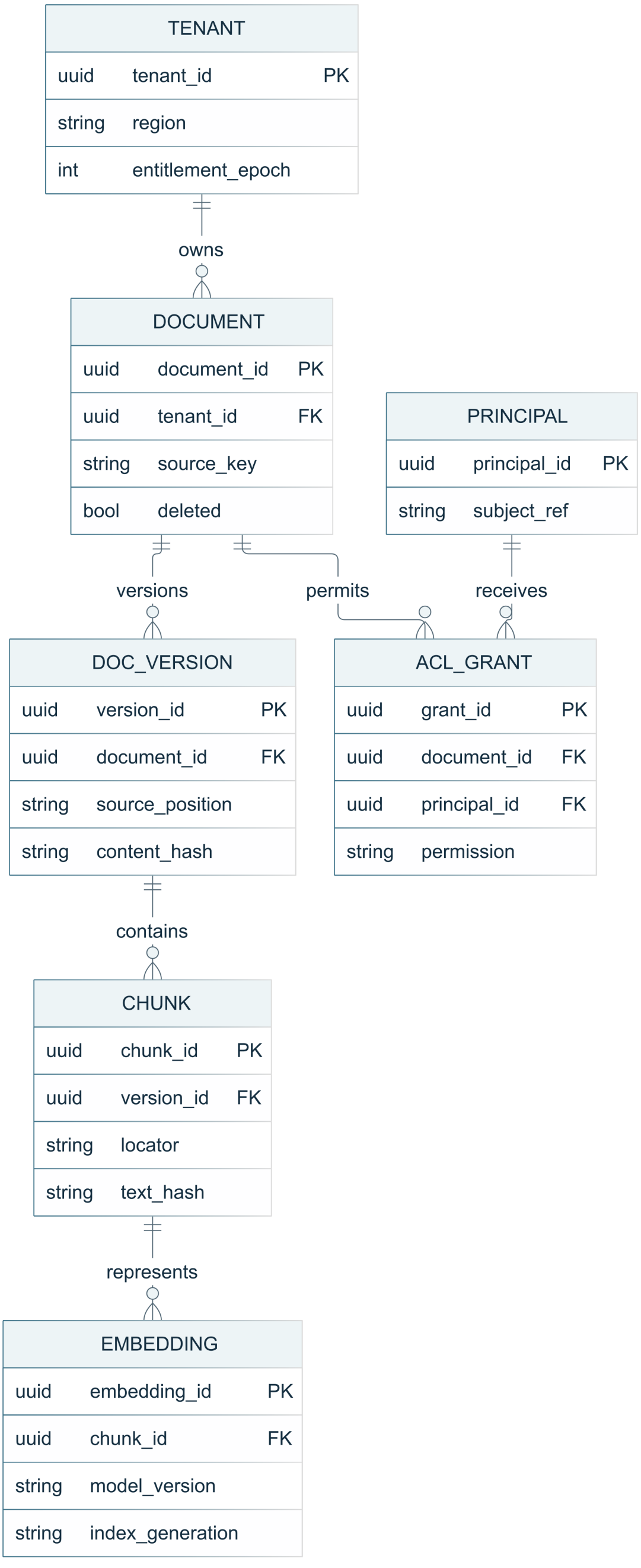


Figure 7. Knowledge and access ER model. Conceptual keys are shown for readability. Physical schemas must enforce tenant-consistent composite references; production authorization may additionally resolve groups, roles, attributes, and deny rules.

13.2 Tenant isolation in physical schemas

Use `(tenant_id, entity_id)` composite keys or unique constraints and tenant-consistent foreign keys where appropriate. Query predicates alone are easy to omit; add row-level security, repository-layer enforcement, and isolation tests as defense in depth. Partitioning or separate databases may be justified for stronger customer isolation, regulatory requirements, or noisy-neighbor control. Keep service-account bypass privileges minimal.

Use immutable source-version and action identifiers, UTC timestamps, explicit status enums, and optimistic version fields. Carry tenant ID through queue envelopes and object metadata but verify it against trusted context before reading a resource. Encrypt sensitive payloads; keep searchable metadata minimal. Maintain group membership and policy-version history needed to explain access decisions without keeping unnecessary personal data forever.

Figure 8. Execution and audit ER model.

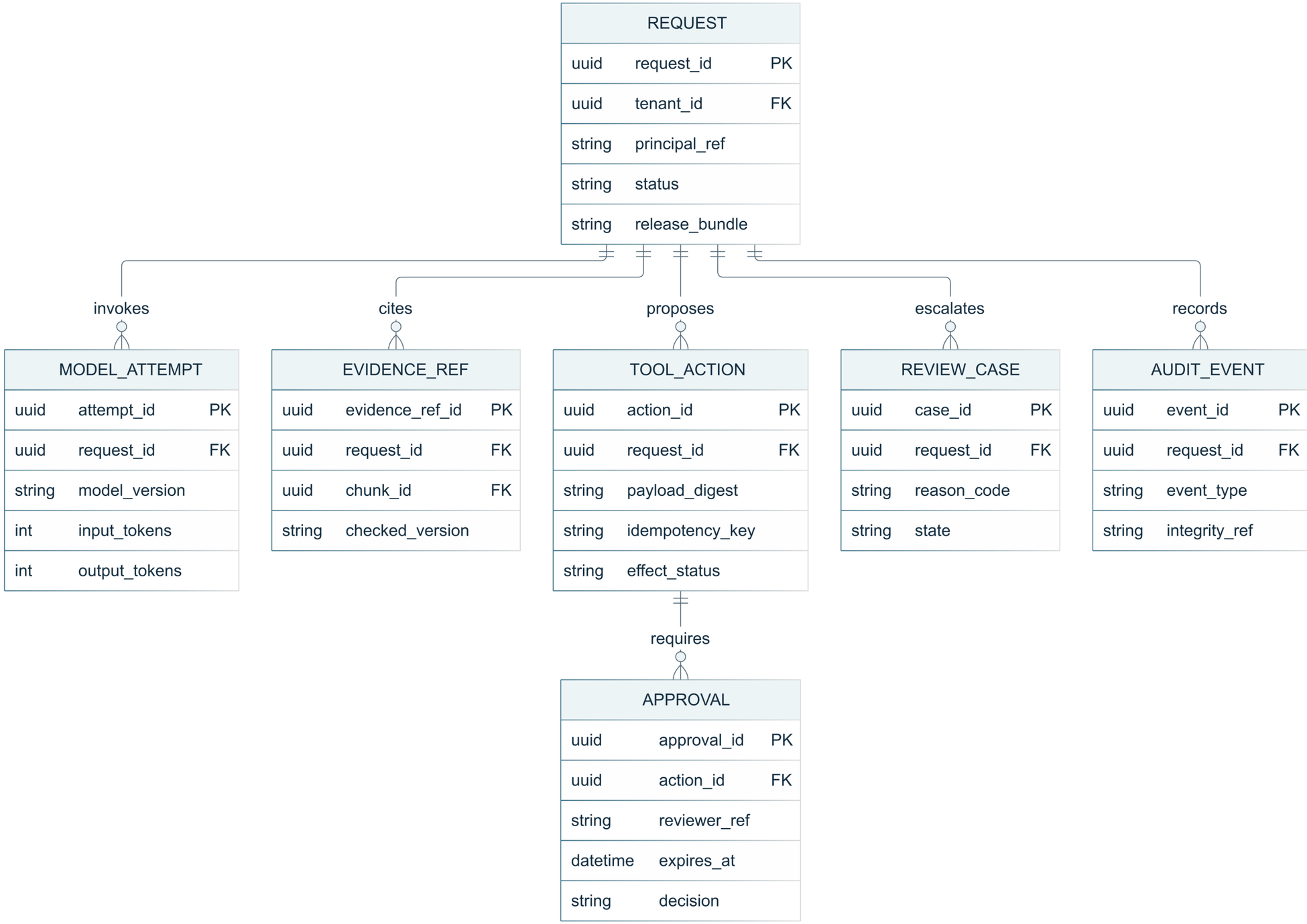


Figure 8. Execution and audit ER model. An approval references an immutable action digest. Multiple approvals support separation of duties. Evidence references connect to Figure 7; detailed retention and usage fields are omitted from this conceptual view.

13.3 API and event contracts

Version request and response schemas explicitly. Distinguish `answered`, `refused`, `needs_clarification`, `pending_review`, `failed`, and `cancelled`; do not force every outcome into an answer string. Include machine-readable reason codes, authorized citations, and a trace reference suitable for support. Do not expose provider credentials, raw prompts, or internal policy text in errors.

An ingestion event should include `event_id`, `tenant_id`, `source_id`, `entity_id`, `operation`, `source_version`, `schema_version`, `occurred_at`, and a payload reference or digest. Validate the producer identity and event schema. Use backward-compatible additive changes when possible; breaking changes need coordinated readers and replay policy. Maintain an inbox ledger for consumed events and an outbox for committed state transitions.

For idempotency records, enforce a unique tenant/principal/operation/key constraint, bind the payload hash, and store status and canonical result reference. Set retention to exceed the maximum client retry and reconciliation period. Expiry should be a deliberate product contract; expiring a key does not mean an old external effect disappeared.

14. Deployment, security governance, and recovery

14.1 Version the complete AI release

Package application code, system prompts, tool schemas, model routing, guardrail policies, embedding configuration, retrieval ranking, index manifest, and evaluator versions as a release bundle. Pin dependencies and record artifact digests. A prompt edit or retrieval change can alter behavior as much as a code deployment, so it needs review, evaluation, canarying, and rollback.

Use isolated development, staging, and production identities and datasets. Production data should not be copied into developer traces without an approved transformation and purpose. Deploy with infrastructure as code, managed secrets, workload identity, encryption at rest, transport encryption, and restricted egress. Scan dependencies and images, validate model artifact provenance, and track software bills of materials where required.

14.2 Regional availability and control-plane safety

Distribute stateless services and durable stores across availability zones where the platform supports it. Replicate queue and database state according to measured durability requirements. Route tenants only to approved regions. Cross-region failover requires compatible indexes, policies, secrets, audit handling, and data-processing eligibility; a healthy model endpoint alone is insufficient.

Cache signed, versioned, last-known-good configuration for serving during control-plane outages, with explicit expiry and revocation semantics. Unknown authorization, expired privileged policy, or unavailable approval state should block protected actions. Less risky static content may have a documented degraded mode. Prevent a bad control-plane rollout from reaching every region simultaneously.

Failure domain	Safe continuity pattern	Recovery prerequisite
API pod or worker	Retry safe requests; resume leased jobs	Durable state, idempotency, cancellation, and fencing.

Failure domain	Safe continuity pattern	Recovery prerequisite
Availability zone	Shift eligible traffic to surviving zones	Spare capacity and replicated state tested under load.
Model provider	Circuit breaker and approved alternative	Equivalent policy eligibility and evaluated behavior.
Search index	Rebuild or switch a validated generation	Source snapshots, manifests, ACL overlay, and restore test.
Primary region	Approved regional failover or controlled outage	Data-residency approval, measured replication lag, and reconciled action state.
Policy or identity dependency	Restricted degraded mode or fail closed	Bounded cached trust material; no bypass of required current authorization.

14.3 Backup, restore, and deletion

Set recovery point objectives (RPO: tolerated data loss) and recovery time objectives (RTO: tolerated restoration time) per store. An illustrative critical job ledger might target RPO below 1 minute and RTO below 30 minutes; a rebuildable search index might accept a longer restoration time. These are requirements to prove in drills, not guarantees from naming a replication feature.

Back up authoritative data, configuration, prompts, manifests, secrets through the approved secret-backup mechanism, and enough source material to rebuild derived stores. Encrypt backups and restrict restore privileges. Regularly restore to an isolated environment, validate record integrity, and execute representative workflows. Test recovery when the latest snapshot is unusable.

Deletion must account for backups and legal holds. Where immutable backups cannot be selectively rewritten, document retention expiry and enforce a deletion ledger on restore before serving data. Do not claim immediate physical erasure across all historical backups. Reapply tombstones, ACL revocations, and policy changes after restore; rolling back an index must never resurrect access.

14.4 Retention, audit, and accountability

Define retention by data class and purpose for raw files, parsed chunks, embeddings, prompts, responses, telemetry, approvals, and review evidence. Store audit records with tamper-evident controls and restricted append/read privileges, but minimize personal content. Keep authorized legal holds explicit and auditable rather than silently overriding deletion everywhere.

Assign ownership: platform owns admission and model gateways; data engineering owns freshness and lineage; application teams own task correctness; security owns policy boundaries and incident containment; product/domain owners define acceptable outcomes; operations owns human review capacity. Cross-functional reviews must resolve conflicting goals such as instant revocation versus offline operation before implementation.

Acceptance evidence: rollback restores a coherent release bundle; a zone or provider failure does not violate policy; backups restore successfully; tombstones survive rollback; credentials can be rotated; operators can disable tools, a model route, or semantic caching independently.

15. Reference implementation patterns

15.1 End-to-end request lifecycle

The sequence below follows a cache miss for a buffered knowledge answer. Cache hits skip retrieval and generation only after their scope and dependencies are validated. Tool actions use the durable approval lifecycle in Chapters 6 and 10. Every expensive stage receives the same request deadline and budget envelope.

Figure 9. Buffered request lifecycle.

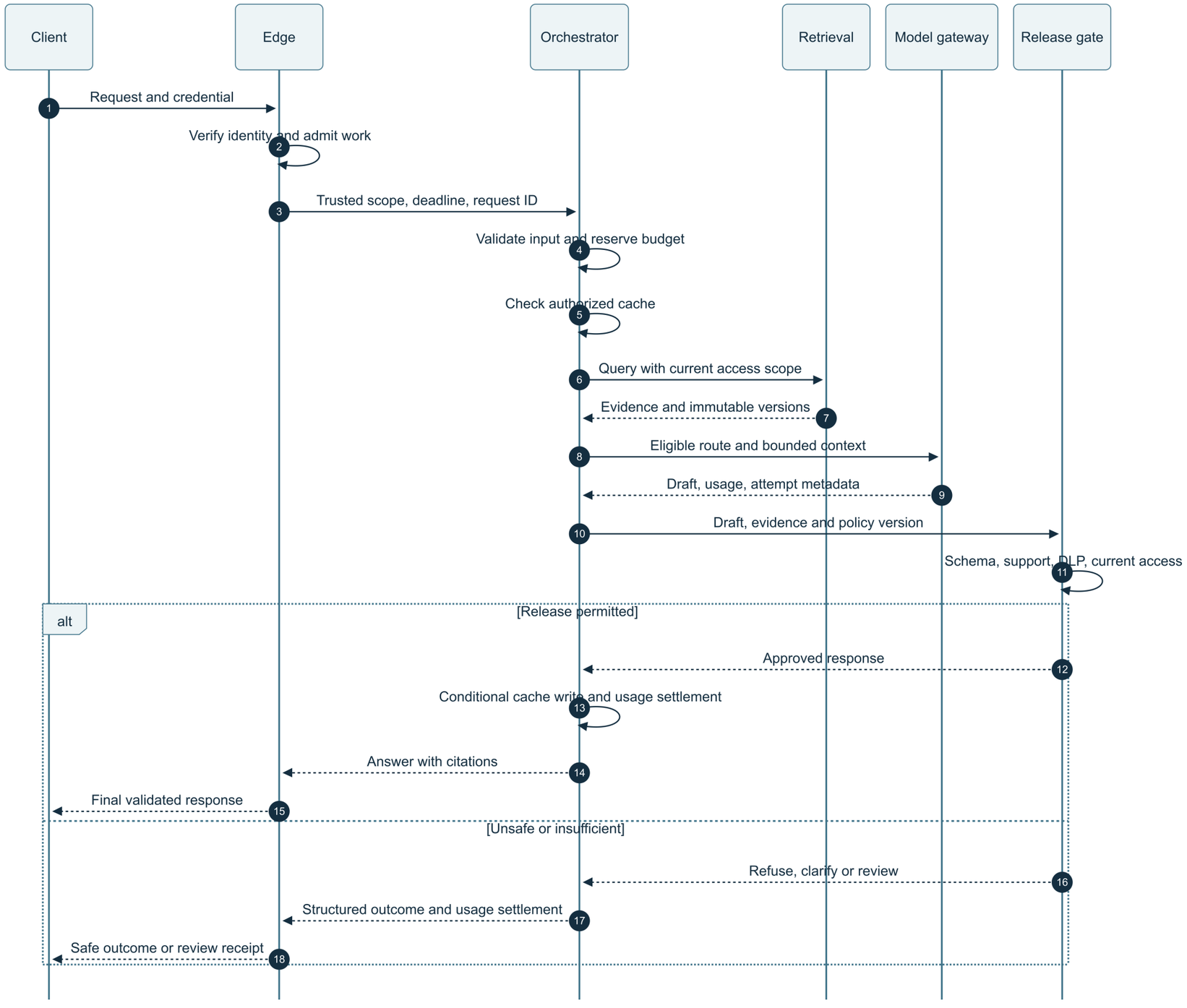


Figure 9. Buffered request lifecycle. Transport errors, cancellation, and budget expiry terminate through the same durable accounting and outcome path; they are omitted from the drawing to keep the main sequence legible.

15.2 A risk-aware policy configuration

This YAML is an application-owned schema sketch, not a configuration for a particular framework. Values illustrate how requirements become explicit. The `internal_qa` route has a 12-second total deadline; the generation limit is a cap within the remaining deadline, not an additional allowance. Only one escalation is allowed, and it must fit the same total budget.

```yaml
policy_version: enterprise-ai-1
routes:
  internal_qa:
    data_class: internal
    allowed_region_set: tenant_approved_regions
    total_deadline_ms: 12000
    max_input_tokens: 12000
    max_output_tokens: 800
    max_model_attempts: 2
    max_total_model_tokens: 24000
    max_request_cost_usd: 0.05
    response_mode: buffered
    evidence_required: true
    semantic_answer_cache: false
    exact_answer_cache_ttl_s: 300
    max_read_tool_calls: 2
    write_tools: []
    checks:
      schema: required
      citation_support: required
      output_dlp: required
      current_authorization: required
  action_proposal:
    response_mode: asynchronous
    durable_state: required
    action_idempotency: required
    current_authorization: required
    approval_bound_to_payload: required
    approval_expiry_s: 900
    fail_if_approval_unavailable: true
```

Budget policy should be enforced centrally or through a trusted shared library with authoritative accounting. A user cannot raise limits by supplying a field with the same name. Requests for higher limits are control-plane changes with ownership and evaluation, not prompt instructions.

15.3 Buffered answer control flow

The following Python-like pseudocode specifies ordering and responsibilities. The services are conceptual interfaces; this is not deployable code. Production implementation additionally needs cancellation propagation, exception mapping, durable usage reconciliation, privacy-safe tracing, and contract tests. A cache exception should become a controlled miss, while a required authorization or DLP dependency failure must not silently pass.

```
async def answer(request, credential):
    ctx = await identity.verify_and_resolve_tenant(credential)
    policy = await policies.resolve(ctx, request.route)
    deadline = Deadline.after_ms(policy.total_deadline_ms)
    clean = await input_gate.validate_and_minimize(request, ctx)

    # Atomic tenant reservation; every attempt draws from this lease.
    budget = await budgets.reserve(ctx, policy.max_request_cost_usd)
    attempts = UsageLedger(request_id=ctx.request_id)
    try:
        scope = await authorization.current_scope(ctx)
        snapshot = await knowledge.active_manifest(ctx.tenant_id)
        key = exact_key(clean, ctx, scope, snapshot, policy)
        candidate = await cache.safe_lookup(key)  # miss on outage
```

```
        if candidate and not await dependencies.current(candidate, ctx):
            candidate = None
        if candidate is None:
            evidence = await retrieval.search(
                clean, scope=scope, generation=snapshot,
                deadline=deadline
            )
            await authorization.check_evidence(ctx, evidence)
            if not evidence.sufficient_for(clean):
                return outcome.needs_clarification_or_abstains()

            plan = router.choose_eligible(clean, ctx, policy)
            candidate = await models.generate_bounded(
                plan, clean, evidence, deadline=deadline,
                budget=budget, usage_ledger=attempts
            )

        checked = await output_gate.validate(candidate, ctx, policy)
        if not checked.allowed:

            return await safe_outcome.persist_if_review(checked, ctx)

        # Gate rechecks current access, tombstones, versions and DLP.
        # Cache only when the captured dependency epochs still match.
        released = await release_gate.authorize_and_prepare(checked, ctx)
        await cache.best_effort_conditional_put(key, released)
        return released.public_response
    finally:
        # Unknown provider outcomes remain reserved/pending until reconciled.
        # Persist settlement intent even on cancellation or worker failure.
        await budgets.persist_settlement(budget, attempts)
```

Do not implement durable settlement only as an in-memory `finally` block: process termination may prevent it from running. Persist attempt admission and provider correlation before sending work, and let a sweeper reconcile expired leases, incomplete attempts, and billing exports. Similarly, a real release gate needs a defined consistency contract: serialize release authorization against authoritative entitlement/tombstone versions, minimize the authorization-to-send interval, and stop later stream segments after revocation. No design can retract bytes already delivered.

15.4 Safe tool execution protocol

Persist an immutable action proposal and payload digest first. Evaluate the requesting principal's current permission for the tool, resource, and arguments. If approval is required, match approvals against the exact digest, approver roles, expiry, and resource preconditions. Atomically claim the action with a lease and fencing token. Send a stable idempotency key to a downstream API that supports the required semantics.

Record success with the downstream operation ID. If the response is lost or times out, mark the effect `unknown` and query the authoritative downstream status using the idempotency or business key. Do not blindly retry a non-idempotent mutation. If status cannot be determined, route to an operator. For APIs without idempotency support, use a domain-specific uniqueness constraint or a reconciliation workflow; otherwise keep the operation manual.

15.5 Cache and ingestion consistency sketches

Construct a cache digest over canonical JSON containing the full scoped context described in Chapter 5. Capture dependency epochs before generation; compare them again at release and cache write. Store provenance with the answer, not only its text. If a cached answer has no usable dependency metadata, it is ineligible for protected enterprise reuse.

An ingestion consumer transaction should check whether `event_id` is already in its inbox, compare the source entity version against the stored version, apply only newer state, record the event, and write downstream outbox

work. Commit before acknowledging delivery. External index writes run from the outbox and are independently idempotent. Publication waits for the required stores; failure leaves the previous coherent generation active unless it is revoked or deleted.

16. Testing, release gates, and rollout

16.1 Test boundaries, not just happy answers

Test layer	Representative cases	Required evidence
Unit and property tests	Canonical cache keys, schema bounds, budget arithmetic, version ordering	Invariants hold across randomized edge cases.
Contract tests	Provider errors, usage formats, tool schemas, queue envelopes	Adapters preserve deadlines, error semantics, and version compatibility.
Integration tests	ACL-filtered RAG, source deletion, review resume, citation download	The complete authorization chain is enforced.
Adversarial tests	Indirect injection, cross-tenant probes, encoded secrets, SSRF	No tested privilege boundary is crossed; failures block release.
Quality evaluation	Representative tasks and difficult slices	Candidate meets predeclared correctness, support, and abstention criteria.
Load and soak	Burst, long outputs, cold cache, slow clients, queue buildup	SLOs, fairness, memory behavior, and spend remain within limits.
Fault injection	Provider timeout, worker crash, lost tool reply, delayed CDC	Safe recovery, no duplicate effects, and useful operator signals.
Recovery drills	Backup restore, index rollback, zone failure	Measured RPO/RTO and preserved tombstones and approvals.

A mock provider proves adapter logic, not real-world quality or latency. Combine deterministic tests with a controlled evaluation environment and representative load against eligible deployments. Disable outbound mutations or use isolated test accounts when replaying traces. Sanitization must preserve the structure that caused the original bug without exposing production secrets.

16.2 A concrete release gate

For an illustrative first production release, require all mandatory schema and authorization tests to pass; zero known cross-tenant disclosures or unauthorized tool effects; no material regression on high-risk evaluation slices; and latency/cost within the route budget at the agreed traffic mix. Set quality thresholds with domain owners. For example, a team may require a lower confidence bound above a chosen answer-correctness target and independently cap unsupported material claims. Do not copy a universal percentage without considering severity and sample size.

Reviewers sign off a release manifest containing code and configuration digests, dataset version, evaluator versions, test results, known limitations, risk exceptions with expiry, rollback target, and on-call owner. Time-limited exceptions need compensating controls; security invariants must not be waived casually to meet a date.

16.3 Progressive deployment

Run the candidate offline first. Then shadow a privacy-approved sample through read-only execution with no user-visible results and no side effects. Compare quality, retrieval, latency, and cost. Canary a small stable cohort, for example 1%, then 5%, 25%, and broader rollout only after predefined observation windows and minimum sample sizes. Percentages are illustrative; low traffic may require longer windows to reveal regressions.

Use tenant- or conversation-stable assignment when consistency matters. Avoid changing model behavior mid-workflow or invalidating an approval by swapping tool schemas. Automatically halt or roll back on hard safety failures and declared SLO burn thresholds. Rollback must restore a coherent model/prompt/retrieval/policy bundle while retaining current deny rules and deletion records.

16.4 Production readiness checklist

- **Product:** supported tasks, unsupported tasks, risk tiers, user-visible outcomes, and escalation behavior are documented.
- **Identity and security:** tenant derivation, downstream authorization, secret handling, egress restrictions, injection tests, and permission revocation are verified.
- **Data:** owners, source contracts, lineage, ACLs, freshness, deletion, tombstones, replay, and index rollback are working.
- **Serving:** deadlines, quotas, cost reservations, concurrency limits, fallback eligibility, cancellation, and streaming release policy are enforced.
- **Actions and review:** idempotency, uncertain-effect reconciliation, bound approvals, reviewer access, queue capacity, and expiry are tested.
- **Quality:** a held-out dataset, slice metrics, calibrated judges, human review, and explicit abstention tests exist.
- **Operations:** dashboards, sanitized traces, audit durability, alerts, kill switches, runbooks, on-call ownership, and restore drills are complete.
- **Governance:** release evidence, processing and retention rules, vendor assessment, incident responsibility, and exception expiry are approved by accountable owners.

17. Failure playbooks and architecture decisions

17.1 Provider latency or error spike

Detect: rising queue wait, timeouts, provider errors, and error-budget burn. **Contain:** open the affected circuit, cap concurrency, reject excess work predictably, and route only to approved alternatives with available budget. **Investigate:** distinguish quota exhaustion, regional failure, long-context changes, and application retry amplification. **Recover:** probe with controlled traffic, ramp gradually, and reconcile incomplete usage. **Prevent:** maintain headroom, adapter tests, and a tested fallback matrix. Retrying at multiple layers can multiply load; assign retry ownership to one layer and bound the total attempts [11].

17.2 Cross-tenant leak or permission failure

Detect: confirmed unauthorized evidence, response, cache hit, or tool access. **Contain:** disable the affected route or cache namespace, revoke exposed credentials if relevant, and stop protected actions. Preserve sanitized forensic references and restrict access to necessary raw evidence. **Investigate:** identity derivation, missing predicates, entitlement caching, citation download paths, and stale grants. **Recover:** invalidate affected derivatives, verify the corrected boundary with explicit negative tests, and follow the organization's incident notification process. Treat this as a security incident even if the prose answer happened to be harmless.

17.3 Stale answers after source or policy updates

Detect: answer provenance predates an authoritative change or freshness lag breaches its objective. **Contain:** bump the relevant epoch, disable semantic reuse, and apply a deny or freshness gate for affected resources. **Investigate:** missing invalidation events, dependency-index gaps, out-of-order CDC, publication races, and stale entitlement caches. **Recover:** replay from a safe checkpoint, reconcile source versions, then republish a coherent generation. Re-run the exact stale-answer case as a regression test.

17.4 Duplicate or uncertain business action

Detect: repeated business effects, inconsistent idempotency records, or a tool timeout with no confirmed outcome. **Contain:** pause the affected action type and retain the proposal, approvals, and downstream correlation IDs. **Investigate:** client retries, worker lease races, key scope, and downstream idempotency retention. **Recover:** query authoritative state and reconcile; use an explicitly authorized compensating action if appropriate. Never resolve uncertainty by repeatedly issuing the same non-idempotent mutation.

17.5 Cost surge, agent loop, or cache stampede

Detect: spend slope, tokens per task, step count, or model-call amplification exceeds expected bounds. **Contain:** enforce hard budget caps, disable the looping route, coalesce safe duplicate work, and shed low-priority demand. **Investigate:** lost cache namespace, prompt growth, recursive tools, retry stacking, longer outputs, and judge sampling. **Recover:** restore evaluated settings, reconcile charges, and re-enable incrementally. Do not remove output safety checks as the first cost-saving response.

17.6 Index corruption or ingestion backlog

Detect: failed integrity checks, embedding dimension errors, count divergence, or a growing oldest-event age. **Contain:** stop publishing damaged generations, preserve the last authorized coherent version, and prioritize deletes and revocations. **Investigate:** parser changes, connector positions, poison documents, write quotas, and partial publication. **Recover:** rebuild or roll back using a validated manifest, reapply tombstones, and verify retrieval slices before increasing traffic.

17.7 Review queue outage or backlog

Detect: urgent cases exceed age thresholds, leases repeatedly expire, or reviewers cannot load evidence. **Contain:** pause actions requiring approval, show an honest pending status, and apply the documented expiry policy. **Investigate:** staffing, permission errors, an overly sensitive trigger, or a release that changed escalation rates. **Recover:** restore reviewer access, prioritize by severity and age, revalidate stale cases, and measure completion rather than merely queue length.

17.8 Architecture decision record template

For consequential choices, write a concise ADR with: decision and date; problem and workload; security and consistency constraints; alternatives; measured evidence; selected option; consequences; owner; rollback or exit strategy; and a revisit trigger. Examples include semantic caching eligibility, shared versus isolated indexes, buffered versus streamed delivery, managed versus self-hosted inference, and workflow versus agent autonomy.

Decision	Prefer the simpler option when	Revisit when
Exact versus semantic answer cache	Repeated queries already match exactly or errors are costly	Evaluated paraphrase demand makes safe semantic reuse valuable.
Shared versus tenant-isolated index	Filters and operational isolation meet proven requirements	Large tenants, policy constraints, or noisy neighbors justify separation.
Workflow versus agent	Steps and allowed effects are known	New tasks need adaptive planning and measured benefits exceed complexity.
Hosted versus self-managed model	Operational simplicity and approved processing fit	Sustained scale, sovereignty, or specialized capability justifies owning inference.
Sync versus async execution	The entire validated path fits the interactive deadline	Research, tools, or review have long or variable completion time.

18. Best practices and delivery roadmap

18.1 The practices that survive model changes

Build authorization, budgets, state, and release policy as application capabilities. Keep prompts and models replaceable behind tested adapters, but do not flatten away meaningful differences in tools, usage accounting, or safety behavior. Improve retrieval and evidence quality before compensating with ever larger prompts. Use the least autonomous workflow that meets the user need, and expand authority only with measurable benefit and bounded consequences.

Optimize the entire task: answer correctness, user effort, latency, cost, and operational burden. A small model plus repeated repairs can cost more than a strong model once. An aggressive semantic cache can improve speed while damaging trust. A human review gate without staffing is an indefinite queue. A fallback that violates residency is a new failure, not resilience.

Make uncertainty explicit and actionable. Good outcomes include a well-supported answer, a targeted clarification, an appropriate refusal, or an honest review receipt. Preserve provenance and exact action records. Use incidents and reviewed failures to improve datasets, tests, and architecture, while respecting the original data-use constraints.

18.2 A staged delivery plan

Stage	Deliverable	Exit condition
Discover	Task definition, risk tiers, source inventory, threat model, SLO draft	Owners agree on supported outcomes and non-negotiable boundaries.
Establish baseline	One model, explicit workflow, authorized retrieval, buffering, traces	Representative held-out tasks are measurable end to end.
Harden	Quotas, budgets, deletion, revocation, replay, output checks, runbooks	Boundary and recovery tests pass; on-call can contain failures.
Optimize	Exact caching, hybrid ranking, eligible model tiers, capacity tuning	Demonstrated improvements without unacceptable slice regressions.
Extend	Semantic reuse, bounded agents, consequential tools, human review	Each new capability has an owner, evaluation evidence, and an independent kill switch.
Operate	Canary releases, ongoing evaluation, restore drills, cost and source reviews	Quality, safety, freshness, and reliability remain within agreed objectives.

18.3 Final design-review questions

1. Can an unauthorized document enter the context through a cache, citation, memory, tool, or fallback path?

2. What happens when a source changes, a permission is revoked, or a user disconnects during generation?

3. Can every retry and agent step be bounded and attributed to a budget and policy decision?

4. How does the system distinguish a failed action from an action whose outcome is unknown?

5. Can a reviewer see the exact effect being approved, and does that approval expire when the proposal changes?

6. Which outputs are checked before release, and what is the accepted residual risk for streaming?

7. Can the team reproduce a bad outcome from versions and evidence without exposing raw secrets?

8. Have cold-cache, noisy-tenant, provider-loss, zone-loss, and restore scenarios been tested?

9. Do measured quality and cost reflect complete tasks, including retries and human work?

10. Can an operator stop one dangerous capability quickly without dismantling the entire service?

An enterprise AI system is ready for sustained operation when these questions have concrete, tested answers and accountable owners. Models will change; the surrounding engineering discipline must continue to make their behavior useful, controlled, and observable.

19. Glossary

Term	Meaning in this handbook
ACL / RBAC / ABAC	Access-control list / role-based / attribute-based authorization.
Agent	A system in which a model chooses some steps or tool calls within an enforced execution boundary.
ANN	Approximate nearest-neighbor search; faster similarity lookup with a configurable recall trade-off.
CDC	Change Data Capture; propagation of source inserts, updates, and deletes to downstream consumers.
Circuit breaker	A control that temporarily stops calls to a failing dependency and uses limited probes to detect recovery.
Context window	Model-specific capacity for input and generated context; not a guarantee of effective attention to every token.
DLP	Data loss prevention; detection and control of sensitive data leaving an allowed boundary.
Embedding	A vector representation of content for similarity and related operations.
Entitlement epoch	Version of access-relevant state used to detect stale authorization-dependent artifacts.
Fencing token	Monotonic ownership token used to reject writes from stale workers after a lease changes.
Groundedness	Degree to which claims are supported by the supplied evidence; distinct from current-world truth.
HITL	Human-in-the-loop; people review, approve, correct, or escalate within an explicit workflow.
Idempotency	Repeating the same logical operation does not create an additional business effect.
KV cache	Stored attention key/value tensors that reduce repeated inference computation.
nDCG	Normalized discounted cumulative gain; a ranked-retrieval metric using graded relevance.
Outbox / inbox	Transactional records for publishing committed changes / deduplicating consumed events.
PII	Personally identifiable information; one subset of sensitive information.
RAG	Retrieval-augmented generation; providing retrieved evidence to generation at request time.
RPO / RTO	Recovery point objective / recovery time objective; tolerated data loss / restoration duration.
RRF	Reciprocal rank fusion; combines ranked lists without requiring comparable raw scores.
SLI / SLO	Service-level indicator / objective; measured user-relevant behavior / target for that behavior.
Semantic cache	Reuse based on meaning similarity plus explicit eligibility, freshness, and permission checks.
Tombstone	Persistent marker preventing deleted or revoked state from being resurrected by delayed updates or restores.
TTFT	Time to first token; specify whether measured at provider output or user-visible release.

20. References and source notes

The architecture, examples, formulas, checklists, and thresholds in this handbook are engineering synthesis. The following primary references support specific standards and product statements cited in the text. They do not certify this architecture or prescribe its illustrative numeric values. URLs and product documentation can change; pin implementation versions and recheck relevant contracts before deployment. Accessed 10 October 2026. No vendor pricing is asserted in the worked cost example.

1. **NIST - Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1 (July 2024).** Risk identification and lifecycle governance reference. Publication and PDF.

2. **OWASP GenAI Security Project - LLM01:2025 Prompt Injection.** Direct and indirect injection, limits of prevention, and defense-in-depth context. Official guidance.

3. **IETF - RFC 9700: Best Current Practice for OAuth 2.0 Security (January 2025).** OAuth threat model and contemporary security guidance. RFC text.

4. **Debezium - Architecture documentation.** Log-based CDC and connector architecture. Official documentation.

5. **pgvector - Official project documentation.** Exact and approximate vector search, HNSW/IVFFlat, and filtered-query considerations. Project reference.

6. **Elastic - Reciprocal rank fusion.** Product reference for combining ranked retrieval results. Elasticsearch documentation.

7. **OpenTelemetry - Generative AI semantic conventions.** GenAI telemetry vocabulary and convention maturity/migration guidance. Official specification entry point.

8. **LangChain - LangSmith Evaluation.** Offline/online evaluation and dataset/experiment workflow. Official documentation.

9. **Google - The Site Reliability Workbook, Chapter 2: Implementing SLOs.** User-oriented SLIs, objectives, and error budgets. SRE workbook.

10. **Arize - What is Arize Phoenix?** Tracing, evaluation, datasets, experiments, and OpenTelemetry/OpenInference integration. Official documentation.

11. **AWS Builders' Library - Timeouts, retries, and backoff with jitter.** Dependency resilience and retry amplification. Official article.

12. **Mermaid - About Mermaid.** Text-based diagram rendering and reusable source format. Official documentation.

Reuse and compilation notes

The Markdown edition contains the complete text and all nine Mermaid source blocks. The companion diagram package includes each diagram as `.mmd`, `.svg`, and `.png`, plus a rendering manifest. Mermaid 12.1.0 (via Kroki 0.33.0) supplied the layouts; HTML-based labels were normalized to standard SVG text for portable export and rasterized at 3x resolution [12]. The PDF uses larger diagram pages where needed for legibility. It is the fixed-layout reading edition; the Markdown and diagram sources are the editable publication assets. Architecture diagrams are logical views, not network topology or deployment manifests. ER diagrams are conceptual schemas; implement tenant-consistent keys and constraints as described in Chapter 13.

To republish, edit the Markdown, render the Mermaid blocks with a pinned compatible renderer, and use a Markdown publishing system that supports tables, fenced code, and generated contents. Keep captions with figures, repeat table headers across pages, and verify every rendered page. Do not treat a successful document build as evidence that a proposed AI system has passed the production tests in Chapter 16.